

Contents

제 1 장, 변수는 왜 흩어져 있는가.	3
1.1, 야생의 변수가 나타났다!	3
제 2 장, 서로 관련된 데이터를 묶어보자.	5
2.1, data class란 무엇인가?	5
2.2, 건축업자가 되는 법	6
2.3, 점 하나의 위대함	7
2.4, 그래서 우리가 뭘 해결한 거죠?	9
제 3 장, 구조체에 함수를 붙여보겠습니다.	9
3.1, 함수들이 밖에서 서성이고 있습니다	10
3.2, 그냥 안에 넣으면 안 돼요?	11
3.3, 데이터가 갑자기 행동하기 시작했다	12
3.4, 잠깐, 그런데 아직도 data class잖아요?	13
3.5, 그래서 클래스가 뭔데요?	15
3.6, 잠시 정리하고 갑시다	16
제 4 장, 설계도에서 물건으로.	17
4.1, 설계도에서는 사람이 살 수 없습니다	18
4.2, 인스턴스라는 이름을 붙여봅시다	19
4.3, 대량생산의 시대	19
4.4, 그래서 객체는 대체 뭔데요?	20
4.5, 객체와 인스턴스의 차이를 37초 안에 설명해보겠습니다	21
4.6, 변수는 객체를 가리킵니다	22
4.7, 타입 추론이라는 이름의 눈치게임	23
4.8, 생성자: 객체 건설 담당 부서	23
4.9, 그런데 말입니다	25
4.10, 아무나 만지게 두면 안 되는 것 같습니다	26
제 5 장, player.hp = -999999999.	26
5.1, 아무나 들어오지 마세요	27
5.2, 보기는 하되 만지지는 마세요	28
5.3, getter와 setter가 또 뭔데요?	29
5.4, 그럼 누가 체력을 바꾸나요?	30
5.5, 객체가 자기 일을 알아서 하게 해봅시다	31
5.6, 왜 굳이 이렇게 귀찮게 하죠?	32
5.7, 객체가 자기 상태를 책임진다는 것	32
5.8, 이것을 캡슐화라고 부릅니다	33
5.9, 캡슐화는 private 도배 대회가 아닙니다	34
5.10, 잠시 정리해봅시다	35
제 6 장, 함수가 탄생한 이유는 재사용성을 위해서였다. 그렇다면 클래스도 재사용할 수 있지 않을까? 재사용의 재사용. 그것이야말로 코딩의 극의!!!	36
6.1, 공통점을 위로 올려봅시다	37
6.2, 조상님을 만들어봅시다	38

6.3, 슬라임에게도 조상님이 생겼습니다	39
6.4, 행동도 물려받을 수 있겠죠?	40
6.5, 아무거나 상속하면 안 됩니다	41
6.6, 그런데 플레이어만 다르게 행동하고 싶다면요?	42
6.7, 잠시 정리해봅시다	43
제 7 장, 형용사적 상속	44
7.1, 명사가 아니라 형용사를 만들어봅시다	45
7.2, 플레이어는 Damageable합니다	45
7.3, 문도 Damageable합니다	46
7.4, 그래서 왜 형용사적 상속인가요?	47
7.5, 능력은 여러 개여도 됩니다	48
7.6, 인터페이스는 약속입니다	49
제 8 장, 문맥이 중요한 이유	50
8.1, 같은 주문, 다른 마법	51
8.2, target은 Damageable인데 어떻게 알아요?	52
8.3, 그래서 좋은 게 뭔데요?	53
8.4, 이것이 다형성입니다	54
8.5, 구체적인 것보다 약속을 바라봅시다	55
8.6, 잠시 정리해봅시다	55
제 9 장, 코딩은 레고임	56
9.1, 플레이어가 체력을 상속받아야만 할까요?	57
9.2, 위치도 부품으로 만들어봅시다	58
9.3, 상속은 유전자고 합성은 레고입니다	59
9.4, 그런데 player.health.damage()가 길어요	60
9.5, 코틀린은 위임 문법까지 있습니다	61
9.6, 상속으로 모든 걸 해결하려는 자의 최후	62
9.7, 잠시 정리해봅시다	63
제 10 장, 객체지향에 대하여	64
10.1, 그래서 객체지향이 뭔데요?	65
10.2, 객체지향은 class 많이 쓰기 대회가 아닙니다	66
10.3, 그 유명한 캡슐화, 상속, 다형성	66
10.4, 추상화는 지금 몰라도 되는 것을 치우는 일입니다	67
10.5, 우리는 객체에게 무엇을 맡겼나요?	68
10.6, 객체지향은 종교가 아닙니다	69
10.7, 그래서 우리가 지금까지 한 일은 무엇이였을까요?	69
10.8, 이제야 정의를 드리겠습니다	71
10.9, 끝났나요?	71
10.10, 그런데 코틀린은 아직 많이 남았습니다	72

제 1 장, 변수는 왜 흩어져 있는가.

자, 게임을 하나 만들어 봅시다.

거창한 건 아닙니다. 플레이어 하나가 있고, 적 하나가 있고, 서로 때리면 체력이 깎이는 아주아주... 평범한 RPG죠.

먼저 플레이어의 이름부터 만들어보겠습니다.

```
var playerName: String = "Yuhan"  
// 코틀린에서 변경 가능한 변수는 var로 선언합니다.  
// 타입 지정은 : 뒤에 적고요.
```

체력도 필요하겠네요.

```
var playerHp: Int = 100  
// 코틀린에서는 세미콜론이 없어도 됩니다.
```

레벨도 만들어봅시다.

```
var playerLevel = 1  
// 오른쪽 값을 보고 컴파일러가 Int라고 추론해줍니다.
```

돈도 넣어보고요.

```
var playerGold = 500
```

좌표도 필요하겠죠.

```
var playerX = 10  
var playerY = 20
```

지금까지는 별문제가 없어 보입니다. 처음 몇 줄에서는 타입도 직접 적었다가 나중에는 생략했다는 것 정도를 빼면요.

```
var playerName: String = "Yuhan"  
var playerHp: Int = 100  
var playerLevel = 1  
var playerGold = 500  
var playerX = 10  
var playerY = 20
```

훌륭합니다. 아마도요.

1.1, 야생의 변수가 나타났다!

플레이어만 있다고 보통 RPG라고 부르지는 않죠. 적도 하나 만들어 봅시다.

```
var enemyName = "Slime"  
var enemyHp = 30  
var enemyLevel = 1  
var enemyGold = 5  
var enemyX = 15  
var enemyY = 20
```

이제 좀 게임 같습니다.

플레이어가 슬라임을 공격하게 만들어보죠.

```
enemyHp -= 10
```

슬라임도 양전히 맞고만 있지는 않을 테니 플레이어를 때리게 합시다.

```
playerHp -= 3
```

잘 작동하네요. 이 정도 코드라면 읽는 데 별 어려움도 없습니다.

그런데 슬라임 한 마리 잡고 엔딩 크레딧이 올라오는 RPG는 좀 허전하겠죠. 슬라임을 한 마리 더 만들어봅시다.

```
var enemy2Name = "Slime"  
var enemy2Hp = 30  
var enemy2Level = 1  
var enemy2Gold = 5  
var enemy2X = 15  
var enemy2Y = 12
```

고블린도 하나 추가하고요.

```
var enemy3Name = "Goblin"  
var enemy3Hp = 30  
var enemy3Level = 1  
var enemy3Gold = 5  
var enemy3X = 42  
var enemy3Y = 18
```

RPG라면 보스도 빠질 수 없겠죠.

```
var bossName = "Ancient Dragon"  
var bossHp = 5000  
var bossLevel = 80  
var bossGold = 10000  
var bossX = 500  
var bossY = 300
```

잠깐만요. 뭔가 이상합니다.

우리는 분명 게임을 만들고 있었는데 어느 순간부터 변수 이름 짓기 대회에 참가하고 있네요.

적이 100마리쯤 생기면 대충 이런 꼴이 될 겁니다.

```
enemy97Hp  
enemy98Hp  
enemy99Hp  
enemy100Hp
```

게임을 만들다가 엑셀 파일이 되어가고 있습니다.

그런데 잘 생각해 보면 `playerName`, `playerHp`, `playerLevel`, `playerGold`, `playerX`, `playerY`는 전부 하나의 대상을 설명하고 있죠.

전부 플레이어의 정보입니다.

우리 머릿속에서는 이미 여섯 값이 한 덩어리인데, 코드에서는 서로 아무 관계도 없는 변수 여섯 개로 흩어져 있습니다. 이름 앞에 `player`라는 접두사를 붙여서 같은 가족이라고 우기고 있을 뿐이죠.

그렇다면 정말 같은 가족으로 만들어주면 되지 않을까요?

제 2 장, 서로 관련된 데이터를 묶어보자.

그렇다면 실제로 묶어봅시다.

C를 공부해본 분이라면 `struct`, 그러니까 구조체라는 말을 들어본 적이 있을지도 모르겠네요. 처음 듣는 말이어도 괜찮습니다. 어차피 지금부터 비슷한 필요를 직접 해결해볼 테니까요.

코틀린에는 `struct`라는 별도의 문법이 없습니다. 대신 지금 우리가 원하는 것처럼 여러 데이터를 하나의 묶음으로 표현할 때는 클래스를 사용할 수 있고, 우선은 `data class`를 이용해보겠습니다.

여러 데이터가 한 교실에 모여 있으니 `data class`.

이 책? PDF 뭉치? 아무튼 이 글자덩어리에서는 일단 데이터 교실이라고 생각합시다.

물론 진짜 어원은 아닙니다. 제가 지금 지어냈습니다.

2.1, `data class`란 무엇인가?

조금 전 만들었던 플레이어 데이터를 전부 가져와봅시다.

```
data class Player(  
    var name: String,  
    var hp: Int,  
    var level: Int,  
    var gold: Int,  
    var x: Int,  
    var y: Int  
)
```

(평범한 데이터 클래스. 평범합니다. 아직은요.)

뭔가 많아 보이지만 사실 아까 흩어져 있던 변수들을 괄호 안에 한 데 모아놓은 것뿐입니다.

전에는 이랬습니다.

```
var playerName = "Yuhan"  
var playerHp = 100
```

```
var playerLevel = 1
var playerGold = 500
var playerX = 10
var playerY = 20
```

이제는 name, hp, level, gold, x, y가 모두 Player라는 하나의 구조 안에 들어가 있습니다.

즉, 우리는 방금 Player라는 새로운 타입을 정의한 셈이죠.

우주 최초의 반이 탄생했습니다.

...라고 하기에는 아직 조금 이릅니다. 정확히는 반을 어떻게 만들지 적어놓은 설계도에 가깝거든요.

교실 평면도만 들고 와서 “우리 반 완성!”이라고 외치면 학교 측에서 조금 곤란해할 겁니다.

그러니 실제로 하나 만들어보죠.

2.2, 건축업자가 되는 법

Player 하나를 만들어보겠습니다.

```
var player: Player = Player(
    name = "Yuhan",
    hp = 100,
    level = 1,
    gold = 500,
    x = 10,
    y = 20
)
```

한 줄에 Player가 두 번 나오고 player까지 하나 더 나왔습니다. 처음 보면 일부러 사람 헛갈리라고 만든 문법처럼 느껴질 수도 있죠.

조금 뜯어봅시다.

```
var player: Player = Player(...)
```

왼쪽부터 보면,

```
var player: Player
```

player라는 변수를 선언했고, 그 변수의 타입은 Player라고 적혀 있습니다.

오른쪽에서는,

```
Player(...)
```

우리가 조금 전에 정의한 Player의 구조를 이용해서 실제 데이터가 들어간 Player 하나를 만들고 있죠.

지금은 대략 이렇게 생각하면 충분합니다.

Player -> Player가 어떤 구조인지 정의한 것
Player(...) -> 그 구조를 이용해서 실제 하나 만들기
player -> 만들어진 것을 가리키는 변수

정확한 이름과 객체가 메모리에서 어떻게 다뤄지는지는 조금 뒤에 다시 이야기하겠습니다.

지금 여기서 다 설명해버리면 4장에서 제가 할 일이 없어집니다. 저자에게도 분량은 필요하죠.

2.3, 점 하나의 위대함

이제 player를 만들었으니 그 안의 값도 사용해야겠죠.

방법은 별것 없습니다. .을 찍으면 됩니다.

```
println(player.name)
println(player.hp)
println(player.level)
```

player.name은 player가 가지고 있는 name을 뜻하고, player.hp는 player가 가지고 있는 hp를 뜻합니다.

아까는 이런 식으로 썼죠.

```
var playerName = "Yuhan"
var playerHp = 100
var playerLevel = 1
```

이제는 변수 이름 안에 player라는 문맥을 매번 옥여넣을 필요가 없습니다.

```
player.name
player.hp
player.level
```

훨씬 보기 좋네요.

점 하나가 세상을 구했습니다. 정확히는 세계 평화 대신 변수 이름 짓기 대회에서 우리를 탈출시켜준 정도지만, 그것만 해도 상당한 업적입니다.

값을 수정하는 방법도 같습니다.

```
player.hp -= 10
player.gold += 100
player.level += 1
```

이제 코드를 읽는 사람은 변수 이름을 해독해서 이것이 누구의 값인지 추론할 필요가 없습니다.

```
player.hp -= 10
```

이라고 되어 있으면 “플레이어의 체력이 줄었구나.”라고 읽으면 되고,

```
player.gold += 100
```

이라고 되어 있으면 “플레이어가 돈을 얻었구나.”라고 읽으면 되죠.

우리가 머릿속에서 생각하던 구조가 코드에도 나타나기 시작한 겁니다.

```
Player
├ name
├ hp
├ level
├ gold
├ x
└ y
```

그리고 이 구조를 여러 번 이용할 수도 있습니다.

```
var player1 = Player(
    name = "Yuhan",
    hp = 100,
    level = 1,
    gold = 500,
    x = 10,
    y = 20
)
```

```
var player2 = Player(
    name = "Mina",
    hp = 120,
    level = 3,
    gold = 200,
    x = 30,
    y = 15
)
```

이제 더 이상,

```
player1Hp
player2Hp
player3Hp
```

같은 이름을 계속 발명할 필요가 없습니다.

```
player1.hp
player2.hp
```

이면 충분하죠.

둘 다 이름은 hp지만 앞에 있는 player1, player2가 이미 누구의 체력인지 문맥을 알려주고 있으니까요.

설계도 하나로 건물을 여러 채 지었습니다. 슬슬 부동산 개발업자가 된 기분도 드네요.

다행히 취득세도 없고 청약 경쟁률도 없습니다. 메모리는 조금 먹습니다.

세상에 완전히 공짜인 것은 잘 없죠.

2.4, 그래서 우리가 뭘 해결한 거죠?

처음에는 플레이어 하나를 표현하기 위해 변수 여섯 개를 따로 관리해야 했습니다.

```
playerName  
playerHp  
playerLevel  
playerGold  
playerX  
playerY
```

그리고 플레이어가 하나 늘어날 때마다 새로운 변수 여섯 개를 또 만들었죠.

지금은 Player라는 구조를 한 번 정의한 뒤,

```
var player1 = Player(...)  
var player2 = Player(...)  
var player3 = Player(...)
```

처럼 필요한 만큼 만들 수 있습니다.

관련된 데이터가 한곳에 모였고, 서로 다른 대상의 데이터가 뒤섞일 가능성도 줄었습니다.

데이터 정리는 어느 정도 끝난 셈이죠.

그런데 플레이어라는 존재가 데이터만 가지고 있을까요?

체력이 있고, 돈이 있고, 좌표가 있는 것만으로는 RPG 주인공 치고는 조금 심심합니다. 때리기도 해야 하고, 맞기도 해야 하고, 회복도 해야 하고, 움직이기도 해야겠죠.

그러면 그런 행동을 담당하는 함수는 어디에 두는 게 좋을까요?

데이터를 전부 Player 안에 넣었으니, 함수도 그냥 같이 넣으면 안 될까요?

다음 장에서 실제로 해봅시다.

제 3 장, 구조체에 함수를 붙여보겠습니다.

지난 장에서 우리는 서로 관련된 데이터를 한곳에 모았습니다.

```
data class Player(  
    var name: String,  
    var hp: Int,  
    var level: Int,  
    var gold: Int,  
    var x: Int,  
    var y: Int  
)
```

이제 플레이어에 관한 데이터는 전부 Player라는 교실 안에 들어가 있죠.

그런데 플레이어는 가만히 앉아서 데이터만 보관하는 서랍장이 아닙니다. 때리기도 하고, 맞기도 하고, 회복도 하고, 돈도 벌고, 가끔은 벽에 끼기도 하죠.

마지막 것은 저희 책임이 아닙니다. 물리 엔진 개발자의 책임입니다.

본인이 물리 엔진까지 개발했다고요? 그렇다면 과거의 자신을 탓하세요. 현재의 당신은 이미 충분히 고생하고 있으니까요.

3.1, 함수들이 밖에서 서성이고 있습니다

우선 플레이어가 피해를 받는 함수를 하나 만들어봅시다.

```
fun damage(player: Player, amount: Int) {  
    player.hp -= amount  
}
```

사용법은 단순하죠.

```
damage(player, 10)
```

회복도 만들어봅시다.

```
fun heal(player: Player, amount: Int) {  
    player.hp += amount  
}
```

레벨업도 필요하겠네요.

```
fun levelUp(player: Player) {  
    player.level += 1  
}
```

돈을 얻는 함수도 만들어보겠습니다.

```
fun addGold(player: Player, amount: Int) {  
    player.gold += amount  
}
```

각각 따로 보면 아무 문제도 없어 보입니다.

그런데 한곳에 모아놓고 보면 조금 이상하죠.

```
fun damage(player: Player, amount: Int)
fun heal(player: Player, amount: Int)
fun levelUp(player: Player)
fun addGold(player: Player, amount: Int)
```

전부 Player를 받습니다. 전부 Player의 데이터를 읽거나 수정하고, 전부 사실상 Player를 위해 존재하는 함수들이죠.

그런데 왜 애네만 밖에 나와 있을까요?

데이터는 교실 안에 넣어놓고 함수는 전부 복도에 세워놨습니다.

이건 함수 차별입니다. 혹시 함수차별주의자신가요?

저는 아닙니다. 그러니 함수들도 안으로 들여보내 보죠.

3.2, 그냥 안에 넣으면 안 돼요?

됩니다. 놀랍게도 정말 그냥 넣으면 됩니다.

```
data class Player(
    var name: String,
    var hp: Int,
    var level: Int,
    var gold: Int,
    var x: Int,
    var y: Int
) {
    fun damage(amount: Int) {
        hp -= amount
    }

    fun heal(amount: Int) {
        hp += amount
    }

    fun levelUp() {
        level += 1
    }

    fun addGold(amount: Int) {
        gold += amount
    }
}
```

아까 밖에 있던 함수들을 Player 안으로 옮겼습니다.

그리고 눈치채셨겠지만 함수의 매개변수에 있던,

player: Player

도 사라졌죠.

이제 함수 자체가 이미 Player 안에 있기 때문입니다.

예를 들어,

```
fun damage(amount: Int) {  
    hp -= amount  
}
```

에서 hp라고 쓰면 아무 player의 체력이나 대충 찾아서 줄이라는 뜻이 아닙니다.

지금 이 함수를 실행하고 있는 *Player*의 hp를 뜻하죠.

사용법도 달라집니다.

예전에는 이렇게 썼습니다.

```
damage(player, 10)  
heal(player, 20)  
levelUp(player)
```

이제는 이렇게 쓸 수 있습니다.

```
player.damage(10)  
player.heal(20)  
player.levelUp()
```

조금 더 자연스럽게 읽히지 않나요?

```
damage(player, 10)
```

은 damage라는 함수에게 player를 넘겨주는 모습이라면,

```
player.damage(10)
```

은 player에게 어떤 행동을 시키는 모습에 가깝습니다.

데이터는 플레이어 안에 있고, 플레이어와 관련된 행동도 플레이어 안에 들어갔습니다.

처음에 변수 여섯 개가 제 각각 흩어져 있던 시절에 비하면 제법 사람 꼴... 아니, 객체 꼴이 나기 시작했네요.

3.3, 데이터가 갑자기 행동하기 시작했다

지금의 Player를 조금 줄여서 보면 이런 모습입니다.

```
data class Player(  
    var name: String,  
    var hp: Int,  
    var level: Int  
) {  
    fun damage(amount: Int) {  
        hp -= amount  
    }  
  
    fun heal(amount: Int) {  
        hp += amount  
    }  
}
```

```
}  
}
```

처음에는,

```
Player  
├─ name  
├─ hp  
└─ level
```

정도였는데 이제는,

```
Player  
├─ name  
├─ hp  
├─ level  
├─  
├─ damage()  
└─ heal()
```

가 되었습니다.

단순히 데이터를 담던 구조가 이제 자기 데이터에 대해 행동할 수 있게 된 거죠.

체력을 가지고 있고, 그 체력을 줄일 수도 있고, 늘릴 수도 있습니다.

구조체 비슷한 것에 팔다리를 달았더니 움직이기 시작했습니다.

프랑켄슈타인 박사도 대충 비슷한 기분이었을지도 모르겠네요. 그 쪽은 컴파일 에러보다 조금 더 심각한 문제가 있었던 것 같지만요.

3.4, 잠깐, 그런데 아직도 **data class**잖아요?

여기서 하나 짚고 넘어가야 합니다.

코틀린의 `data class`도 엄연한 클래스입니다. 함수도 얼마든지 넣을 수 있죠.

```
data class Player(  
    var hp: Int  
) {  
    fun damage(amount: Int) {  
        hp -= amount  
    }  
}
```

이렇게 작성해도 아무 문제 없습니다. 컴파일러도 화내지 않고 경찰도 오지 않습니다.

그러면 왜 이제부터 `data`를 떼려고 할까요?

우리가 처음 data class를 사용했을 때의 관심사는 거의 하나였습니다.

서로 관련된 데이터를 한곳에 묶자.

그래서 이 글자덩어리에서는 데이터 교실이라는 비유까지 사용했죠.

그런데 지금은 Player가 어떤 데이터를 가지고 있는지만 표현하고 싶은 것이 아닙니다. Player가 무엇을 할 수 있는지도 같이 표현하고 싶습니다.

damage
heal
attack
defend
move
levelUp
addGold

그러니 이제부터는 데이터 묶음보다는 하나의 대상이라는 측면에 집중하기 위해 평범한 class를 사용해보겠습니다.

```
class Player(  
    var name: String,  
    var hp: Int,  
    var level: Int,  
    var gold: Int,  
    var x: Int,  
    var y: Int  
) {  
    fun damage(amount: Int) {  
        hp -= amount  
    }  
  
    fun heal(amount: Int) {  
        hp += amount  
    }  
  
    fun levelUp() {  
        level += 1  
    }  
  
    fun addGold(amount: Int) {  
        gold += amount  
    }  
}
```

겉보기에는 data라는 단어 하나를 지웠을 뿐입니다.

물론 실제로는 `data class`가 자동으로 제공하던 `toString()`, `equals()`, `copy()` 같은 기능도 함께 달라집니다. 그 이야기를 여기서 자세히 시작하면 책이 갑자기 옆길로 달려갈 테니, 지금은 그런 차이가 있다는 정도만 알아두죠.

중요한 변화는 문법보다 우리가 이 구조를 바라보는 방식입니다. 처음에는,

관련된 데이터를 한데 묶자.

였다면, 지금은,

하나의 대상이 자신의 데이터와 행동을 함께 가지게 하자.

가 되었습니다.

이렇게 어떤 대상이 가지는 데이터와 행동을 함께 정의하는 틀이 바로 클래스(*class*)입니다.

3.5, 그래서 클래스가 뭔데요?

교과서에서는 흔히 이런 설명을 볼 수 있습니다.

클래스는 객체를 만들기 위한 설계도다.

틀린 말은 아닙니다. 오히려 상당히 괜찮은 비유죠.

그런데 객체지향을 처음 배우는 입장에서는 “아, 클래스가 설계도구나.”까지 이해하고도 여전히 “그래서 왜 필요한데요?”라는 질문이 남기 쉽습니다.

우리는 그 이유부터 거꾸로 만들어왔습니다.

처음에는 변수들이 흩어져 있었습니다.

```
var playerName = "Yuhan"
var playerHp = 100
var playerLevel = 1
```

이 값들이 하나의 플레이어를 설명한다는 사실을 깨닫고 관련된 데이터를 묶었습니다.

```
data class Player(
    var name: String,
    var hp: Int,
    var level: Int
)
```

그리고 조금 지나자 그 데이터를 다루는 함수들도 전부 플레이어와 관련되어 있다는 사실이 보였죠.

그래서 그것까지 같이 넣었습니다.

```
class Player(  
    var name: String,  
    var hp: Int,  
    var level: Int  
) {  
    fun damage(amount: Int) {  
        hp -= amount  
    }  
}
```

그러니 지금 단계에서는 이렇게 이해해도 충분합니다.

클래스는 서로 관련된 데이터와 행동을 한데 묶어 어떤 대상의 구조를 정의한 것입니다.

그리고 우리는 이 구조를 이용해서 실제 플레이어도 만들 수 있죠.

```
var player = Player(  
    name = "Yuhan",  
    hp = 100,  
    level = 1  
)
```

만들어진 플레이어는 클래스 안에 정의한 행동도 사용할 수 있습니다.

```
player.damage(10)
```

```
println(player.hp)
```

출력은 다음과 같겠죠.

```
90
```

이제 플레이어는 숫자 몇 개가 우연히 근처에 모여 있는 존재가 아닙니다. 자기 이름과 체력과 레벨을 가지고 있고, 그 체력에 작용하는 행동도 함께 가지고 있습니다.

상당히 객체 같아졌네요.

사실 이미 객체입니다.

다만 여기서 객체라는 말까지 제대로 설명해버리면 다음 장 제목이 상당히 곤란해 집니다. 저자에게도 서사가 필요하니 조금만 참아주세요.

3.6, 잠시 정리하고 갑시다

이번 장에서 한 일은 생각보다 단순합니다.

관련된 데이터를 묶었습니다.

↓

그 데이터와 관련된 함수도 같이 묶었습니다.

↓

하나의 대상을 정의하는 클래스가 생겼습니다.

객체지향이라는 말을 처음 들으면 무언가 거대한 철학이나 고대 비전의 프로그래밍 기법처럼 느껴질 수도 있습니다.

하지만 우리가 지금까지 한 일의 출발점은 꽤 소박하죠.

같이 다니는 것들은 같이 두자.

플레이어의 체력과 이름과 레벨이 같이 다닌다면 한곳에 두고, 그 데이터를 다루는 행동도 늘 플레이어와 함께 다닌다면 같은 곳에 두는 겁니다.

결국 정리정돈이죠.

객체지향의 시작이 의외로 방 청소와 비슷합니다. 물론 제 방은 객체지향적이지 않습니다. 데이터도 행동도 여기저기 흩어져 있고 가끔 필요한 물건의 참조를 잃어버립니다.

GC라도 있었으면 좋겠네요.

이제 다음 장에서는 앞에서 계속 사용하면서도 제대로 이름을 붙이지 않았던 질문을 다뤄보겠습니다.

Player는 설계도라면서요?

그럼 player가 가리키는 자건 대체 뭔데요?

제 4 장, 설계도에서 물건으로.

지난 장에서 우리는 class를 만들었습니다.

```
class Player(  
    var name: String,  
    var hp: Int,  
    var level: Int  
) {  
    fun damage(amount: Int) {  
        hp -= amount  
    }  
  
    fun heal(amount: Int) {  
        hp += amount  
    }  
}
```

이름도 있고, 체력도 있고, 레벨도 있고, 맞기도 하고 회복도 하는 제법 그럴듯한 Player의 구조가 완성됐죠.

그러니 게임을 실행해볼까요?

...

아무것도 없습니다.

네. 플레이어를 아직 안 만들었거든요.

지금까지 만든 것은 플레이어 그 자체가 아니라 플레이어가 어떻게 생기고 행동해야 하는지 적어놓은 정의이기 때문입니다.

어딘가 익숙하지 않나요? 기획서만 200페이지 작성하고 실제 구현은 한 줄도 없는 프로젝트와 비슷합니다.

혹시 찢리셨다면 죄송합니다. 저도 찢렸습니다.

4.1, 설계도에서는 사람이 살 수 없습니다

클래스를 다시 한번 보죠.

```
class Player(  
    var name: String,  
    var hp: Int,  
    var level: Int  
)
```

여기에는 구체적인 이름도 없고 체력이 얼마인지도, 레벨이 몇인지도 정해져 있지 않습니다.

그저 대략,

Player라는 것은 name을 가지고, hp를 가지고, level을 가진다.

정도만 정해져 있죠.

건축 설계도에 방 3개, 화장실 2개, 창문 7개라고 적혀 있다고 해서 거기 들어가 주무시면 곤란합니다. 아무리 고급 용지를 사용해도 설계도는 여전히 종이니까요.

그러니 실제로 하나 지어봅시다.

```
var player = Player(  
    name = "Yuhan",  
    hp = 100,  
    level = 1  
)
```

이제 실제 데이터가 들어 있는 Player 하나가 만들어졌습니다.

건축 기간은 컴퓨터 사양에 따라 조금 다르겠지만 사람 입장에서는 거의 순식간이죠. 이 정도 속도라면 주택 공급 문제를 해결할 수 있을지도 모르겠습니다.

물론 집 대신 객체가 공급됩니다. 거주에는 적합하지 않습니다.

4.2, 인스턴스라는 이름을 붙여봅시다

우리가 방금 작성한 이 부분을 보죠.

```
Player(  
    name = "Yuhan",  
    hp = 100,  
    level = 1  
)
```

Player라는 클래스에 정의된 구조를 바탕으로 실제 데이터를 가진 하나를 만들었습니다.

이렇게 어떤 클래스를 바탕으로 실제 만들어진 하나를 그 클래스의 인스턴스(*instance*)라고 부릅니다.

프로그래밍 공부를 하다 보면 이런 일이 자주 있습니다. 개념 하나를 이해했더니 새로운 이름이 두세 개씩 튀어나오죠.

그래도 이번 것은 비교적 단순합니다.

Player

→ 클래스, 구조의 정의

Player("Yuhan", 100, 1)

→ Player를 바탕으로 실제 하나 생성

→ Player의 인스턴스

여기서 존재론과 형이상학으로 빠질 필요는 없습니다. 아직 객체지향 책임입니다.

철학서는 4권쯤에서 쓰겠습니다. 그런데 아마 안 쓸 겁니다. 귀찮거든요.

4.3, 대량생산의 시대

클래스가 있다는 것은 같은 구조를 여러 번 이용할 수 있다는 뜻이죠.

두 명을 만들어봅시다.

```
var player1 = Player(  
    name = "Yuhan",  
    hp = 100,  
    level = 1  
)
```

```
var player2 = Player(  
    name = "Mina",  
    hp = 80,
```

```
    level = 3  
)
```

player1과 player2가 가리키는 대상은 둘 다 Player 클래스를 바탕으로 만들어졌으므로 둘 다 Player의 인스턴스입니다.

하지만 같은 인스턴스는 아니죠.

```
player1.hp -= 10
```

```
println(player1.hp)  
println(player2.hp)
```

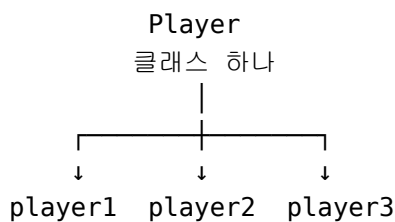
결과는 이렇게 됩니다.

```
90  
80
```

player1이 맞았다고 해서 player2까지 같이 아파하지는 않습니다.

그 정도면 객체지향이 아니라 양자얍힘입니다. 이 글자덩어리의 담당 범위를 명백히 벗어나므로 물리학과에 문의해주세요.

대략 이런 모습이라고 생각하면 됩니다.



설계도는 하나지만 그 설계도를 이용해서 서로 독립적인 여러 대상을 만들 수 있죠.

대량생산의 시대입니다. 하늘에 계신 포드 씨가 흐뭇하게 바라볼 것 같네요.

4.4, 그래서 객체는 대체 뭔데요?

이제 피할 수 없는 단어가 하나 나옵니다.

객체(object)입니다.

객체지향 프로그래밍, Object-Oriented Programming. 무려 장르 이름에 들어 있는 단어인데 4장까지 와서 이제 제대로 설명하네요.

교육과정 설계가 잘못된 것 같다고요? 의도한 겁니다. 아마도요.

지금 단계에서는 객체를 대략 이렇게 생각해도 충분합니다.

실제로 존재하면서 자기 상태를 가지고, 그 상태와 관련된 행동을 수행할 수 있는 하나의 대상.

우리의 player1이 가리키는 실제 Player가 그렇죠.

```
player1.hp  
player1.level  
  
player1.damage(10)  
player1.heal(10)
```

자기 데이터가 있고 자기 행동도 있습니다.
그래서 이것을 객체라고 부를 수 있습니다.
그러면 자연스럽게 질문 하나가 생기겠죠.

객체와 인스턴스는 다른 말인가요?

조금 다르면서도 실제 대화에서는 상당히 겹쳐서 사용됩니다.
좋은 설명이군요. 조금만 더 제대로 말해봅시다.

4.5, 객체와 인스턴스의 차이를 37초 안에 설명해보겠습니다

다음 코드를 봅시다.

```
var player = Player(  
    name = "Yuhan",  
    hp = 100,  
    level = 1  
)
```

여기서 생성된 Player는 하나의 객체입니다.

동시에 이 객체는 Player라는 클래스를 바탕으로 만들어졌으므로 **Player**의 인스턴스이기도 하죠.

둘은 서로 완전히 별개의 물건을 뜻한 다기보다 같은 대상을 어느 관점에서 부르느냐에 가깝습니다.

객체

→ 실제 존재하는 대상 자체에 초점을 둔 표현

인스턴스

→ 그 대상이 어떤 클래스에서 만들어졌는지에 초점을 둔 표현

예를 들어,

저 사람은 사람입니다.

라고 할 수도 있고,

저 사람은 Homo sapiens의 한 개체입니다.

라고 할 수도 있겠죠.

둘 다 같은 존재를 가리키지만 후자는 분류 관계를 훨씬 강하게 드러냅니다.

물론 일상 대화에서 후자를 자주 사용하면 친구가 줄어들 가능성이 있습니다.

프로그래밍에서는 다행히 둘 다 정상적인 표현입니다.

이 정도면 입문 단계에서는 충분합니다. 객체와 인스턴스의 존재론적 차이를 고민하다가 새벽 세 시에 천장을 바라볼 필요는 없습니다.

37초가 지났는지는 모르겠습니다. 측정하지 않았거든요.

4.6, 변수는 객체를 가리킵니다

이 코드를 조금 더 뜯어보죠.

```
var player: Player = Player(  
    name = "Yuhan",  
    hp = 100,  
    level = 1  
)
```

왼쪽에는 이런 것이 있습니다.

```
var player: Player
```

player라는 변수를 선언했고, 그 변수의 타입은 Player입니다.

오른쪽에서는 실제 Player 객체를 하나 만들죠.

```
Player(  
    name = "Yuhan",  
    hp = 100,  
    level = 1  
)
```

그러니까 지금 단계에서는 대충 이렇게 읽을 수 있습니다.

```
var player: Player
```

→ Player를 가리킬 player라는 변수를 만들겠습니다.

```
Player(...)
```

→ Player 객체 하나 만들겠습니다.

=

→ 방금 만든 객체를 player가 가리키게 하겠습니다.

갑자기 대단히 소박해졌네요.

프로그래밍은 한 줄씩 뜯어보면 생각보다 별것 아닌 경우가 많습니다. 문제는 그런 별것 아닌 줄이 14만 줄쯤 모여 있는 경우죠.

여기서 제가 일부러 `player` 안에 객체가 들어 있다고 하지 않고, `player`가 객체를 가리킨다고 표현했습니다.

그 이유를 지금 완전히 설명하려면 참조, 메모리, 객체의 생명주기 같은 친구들이 단체로 교실 문을 열고 들어옵니다. JVM의 메모리 모델까지 끌어오기 시작하면 우리 반 정원이 금방 초과되겠죠.

지금은 `player`라는 이름을 통해 어떤 `Player` 객체에 접근하고 있다 정도로 생각하면 충분합니다.

필요해지면 나중에 다시 데려오죠.

4.7, 타입 추론이라는 이름의 눈치게임

사실 Kotlin에서는 타입을 매번 직접 적을 필요도 없습니다.

```
var player: Player = Player(  
    name = "Yuhan",  
    hp = 100,  
    level = 1  
)
```

물론 이렇게 써도 되지만,

```
var player = Player(  
    name = "Yuhan",  
    hp = 100,  
    level = 1  
)
```

이렇게 써도 됩니다.

컴파일러가 오른쪽의 `Player(...)`를 보고,

오른쪽에서 `Player`를 만들었네요. 그러면 `player`의 타입도 `Player`겠군요.

라고 판단해주기 때문입니다.

아주 눈치가 빠릅니다. 조금 기분이 나쁠 정도네요.

하지만 컴파일러에게 빠져봤자 프로젝트는 빌드되지 않으니 받아들이도록 합시다.

4.8, 생성자: 객체 건설 담당 부서

그런데 여기서 이런 의문이 생길 수도 있습니다.

```
Player(  
    name = "Yuhan",  
    hp = 100,  
    level = 1  
)
```

왜 `Player(...)`는 함수 호출처럼 생겼을까요? 괄호까지 아주 당당하게 붙어 있죠.

새로운 객체를 만들 때 사용하는 것이 생성자(*constructor*)입니다.

이름이 놀라울 정도로 정직합니다. 객체를 생성하니까 생성자죠.

개발자들은 가끔 정말 이름을 잘 짓습니다. 가끔은요.

우리의 클래스를 다시 보겠습니다.

```
class Player(  
    var name: String,  
    var hp: Int,  
    var level: Int  
)
```

`class Player` 바로 뒤에 붙어 있는 괄호 부분은 `Player`의 주 생성자(**primary constructor**) 선언에 해당합니다.

새로운 `Player`를 만들 때 이름, 체력, 레벨을 받아야 한다는 뜻이죠.

그래서,

```
Player(  
    name = "Yuhan",  
    hp = 100,  
    level = 1  
)
```

이라고 쓰면 대충 이런 일을 요청하고 있는 셈입니다.

[`Player` 건설 요청 접수]

```
name = "Yuhan"  
hp    = 100  
level = 1
```

↓

새로운 `Player` 객체 생성

↓

준공

분양가는 0원입니다. 메모리 사용료는 조금 나갑니다.

아무튼 이렇게 해서 우리는 하나의 클래스로 여러 객체를 만들고, 각 객체가 서로 다른 데이터를 가지게 하는 데 성공했습니다.

이 정도면 제법 그럴듯한 RPG가 만들어질 것 같죠.

그런데 문제가 하나 생깁니다.

물론 생깁니다. 이 책에서 문제가 안 생기면 다음 장을 쓸 수가 없거든요.

4.9, 그런데 말입니다

우리 Player를 다시 한번 보죠.

```
class Player(  
    var name: String,  
    var hp: Int,  
    var level: Int  
)
```

hp가 var로 선언되어 있으니 외부 코드에서도 값을 얼마든지 바꿀 수 있습니다.

```
player.hp = 50
```

이 정도는 별문제 없어 보이네요. 공격을 많이 맞았나 봅니다.

```
player.hp = 3
```

죽기 직전이군요. 슬슬 포션을 마시는 편이 좋겠습니다.

그런데 이런 것도 됩니다.

```
player.hp = -10
```

플레이어가 갑자기 생과 사의 경계를 넘어섰습니다.

버그가 조금 있나 보죠. 그런데 level 역시 var이기 때문에 이런 것도 가능합니다.

```
player.level = 999999999
```

자, 여러분은 며칠 동안 레벨링 시스템을 만들었다고 해봅시다.

경험치 테이블을 만들었고, 일정 경험치를 모으면 레벨이 올라가며, 레벨이 오르면 스탯이 증가하고 스킬이 해금되게 만들었습니다.

며칠 동안 열심히 일한 보람이 있겠네요.

그런데 프로젝트 어딘가에서 누군가 이렇게 씁니다.

```
player.level = 999999999
```

여러분의 며칠이 사라졌습니다.

축하드립니다.

4.10, 아무나 만지게 두면 안 되는 것 같습니다

우리가 정말 원했던 것은,

```
player.level = 999999999
```

같은 코드가 아니었죠.

아마 이런 쪽에 더 가까웠을 겁니다.

```
player.gainExp(100)
```

경험치를 얻고, 필요한 만큼 모였다면 레벨이 올라가고, 그 과정에서 스탯도 증가하고 스킬도 해금되는 식입니다.

즉, level이라는 값을 아무 코드나 직접 바꾸게 두면 안 될 것 같습니다.

HP도 마찬가지죠.

```
player.hp = -92834723
```

같은 것이 아무렇지 않게 가능하면 우리 게임의 생명과 죽음에 대한 철학이 필요 이상으로 복잡해집니다.

외부에서는 값을 마음대로 수정하지 못하게 하고, 우리가 정해둔 방법을 통해서만 상태를 바꾸게 하면 어떨까요?

예를 들어 체력은 damage()나 heal()을 통해 바꾸고, 레벨은 경험치 시스템을 통해서만 올라가게 만드는 식으로요.

다행히 그런 기능이 있습니다.

없었다면 이 책이 여기서 갑자기 끝났을 겁니다.

제 5 장, player.hp = -999999999.

지난 장에서 우리는 꽤 심각한 문제를 하나 발견했습니다.

```
player.hp = -92834723
```

```
player.level = 999999999
```

이런 코드가 아무런 저항 없이 실행된다는 것이었죠.

컴파일러도 말리지 않고 IDE도 말리지 않습니다. 운이 나쁘면 코드 리뷰에서도 지나갑니다.

그리고 게임을 실행한 플레이어만 “어?” 하고 죽겠죠.

아니, 체력이 음수니까 이미 죽은 건지 죽음을 초월한 건지 조금 애매하네요.

아무튼 좋지 않습니다.

우리가 원하는 것은 Player의 상태를 아무 코드나 마음대로 바꿀 수 있는 세상이 아닙니다.

체력은 공격을 받거나 회복했을 때 바뀌고, 레벨은 경험치를 얻어서 조건을 만족했을 때 올라가야겠죠.

그러니 일단 문부터 잠가봅시다.

5.1, 아무나 들어오지 마세요

현재 우리의 Player는 이런 모습입니다.

```
class Player(  
    var name: String,  
    var hp: Int,  
    var level: Int  
) {  
    fun damage(amount: Int) {  
        hp -= amount  
    }  
  
    fun heal(amount: Int) {  
        hp += amount  
    }  
}
```

hp가 공개된 var이기 때문에 외부에서 읽을 수도 있고,

```
println(player.hp)
```

직접 쓸 수도 있습니다.

```
player.hp = -999999999
```

문이 활짝 열려 있는 셈이죠.

택배기사도 들어오고, 옆집 사람도 들어오고, 지나가던 코드도 들어와서 가구 배치를 바꿀 수 있습니다.

보안 상태가 조금 심각합니다.

그렇다면 hp를 아예 밖에서 접근하지 못하게 해보면 어떨까요?

코틀린에는 private이라는 접근 제한자가 있습니다.

```
class Player(  
    var name: String,  
    private var hp: Int,  
    var level: Int  
)
```

이제 클래스 바깥에서는 hp에 직접 접근할 수 없습니다.

```
println(player.hp)
```

이것도 안 되고,

```
player.hp = 50
```

이것도 안 됩니다.

문을 아주 훌륭하게 잠갔습니다.

문제는 밖에서 체력 확인조차 할 수 없다는 겁니다.

게임 화면에 체력바를 그리려고 했는데 HP를 읽을 수가 없습니다.
보안은 완벽한데 서비스도 완벽하게 사용할 수 없네요.

보통 이런 것을 좋은 설계라고 하지는 않습니다.

5.2, 보기는 하되 만지지는 마세요

우리가 원했던 것을 다시 생각해보죠.

외부 코드가 체력을 읽는 것까지 막고 싶었던 것은 아닙니다.

```
println(player.hp)
```

HUD를 그릴 때도 필요하고, 상태를 표시할 때도 필요하며, 디버깅할 때도 유용하겠죠.

문제는 이쪽입니다.

```
player.hp = -999999999
```

읽는 것은 괜찮지만 아무나 쓰게 두면 곤란한 겁니다.

그렇다면,

읽는 건 허용하고, 수정은 막자.

라고 하면 되겠죠.

코틀린에서는 상당히 직접적으로 표현할 수 있습니다.

```
class Player(  
    var name: String,  
    hp: Int,  
    var level: Int  
) {  
    var hp: Int = hp  
    private set  
}
```

private set이라고 적혀 있네요.

즉, 외부에서는 값을 읽을 수 있지만 그 값을 설정하는 기능은 클래스 밖에 공개하지 않겠다는 뜻입니다.

```
println(player.hp)
```

이 코드는 사용할 수 있지만,

```
player.hp = 500
```

은 사용할 수 없습니다.

대충 객체가 이렇게 말하는 셈입니다.

구경은 하셔도 됩니다. 다만 손대지는 마세요.

박물관이 되었습니다.

5.3, getter와 setter가 또 뭔데요?

여기서 setter라는 이름이 갑자기 튀어나왔습니다.

프로그래밍은 원래 이런 식입니다. 용어 하나를 설명하기 위해 용어 두 개가 추가됩니다.

이런 코드를 생각해 보죠.

```
player.hp = 50
```

겉으로 보면 그냥 변수에 값을 넣는 것처럼 보이지만, 코틀린의 프로퍼티에는 값을 읽고 쓰는 동작이 연결되어 있습니다.

값을 읽는 쪽을 *getter*라고 부르고,

```
println(player.hp)
```

값을 쓰는 쪽을 *setter*라고 부릅니다.

```
player.hp = 50
```

입문 단계에서는 이렇게 생각하면 충분합니다.

```
player.hp
```

→ 값 읽기

→ *getter*

```
player.hp = 50
```

→ 값 쓰기

→ *setter*

그러니,

```
private set
```

은 *setter*의 접근 범위를 *private*으로 제한한다는 뜻입니다.

밖에서는 *getter*를 통해 값을 볼 수 있지만 *setter*를 이용해 값을 직접 바꿀 수는 없는 거죠.

문은 잠갔는데 창문으로 안을 볼 수는 있습니다.

표현이 조금 수상하네요. 다른 비유를 찾을 걸 그랬습니다.

5.4, 그럼 누가 체력을 바꾸나요?

외부 코드가 체력을 직접 수정하지 못하게 막았습니다.

그러면 체력은 누가 바꿀까요?

Player 안에 있는 행동이 바뀌면 됩니다.

우리는 이미 그런 함수를 만들어놨죠.

```
class Player(  
    var name: String,  
    hp: Int,  
    var level: Int  
) {  
    var hp: Int = hp  
    private set  
  
    fun damage(amount: Int) {  
        hp -= amount  
    }  
  
    fun heal(amount: Int) {  
        hp += amount  
    }  
}
```

이제 외부에서는 체력을 직접 90으로 설정하기보다는,

```
player.hp = 90
```

플레이어에게 10의 피해가 발생했다고 전달할 수 있습니다.

```
player.damage(10)
```

결과만 보면 둘 다 체력을 100에서 90으로 만들 수도 있습니다.

하지만 의미는 꽤 다르죠.

첫 번째는,

너 이제 HP 90이야.

라고 외부에서 결과를 강제로 집어넣는 것이고,

두 번째는,

너 10만큼 피해를 입었어.

라고 객체에게 어떤 일이 일어났는지 전달하는 것에 가깝습니다.

그리고 이 차이가 생각보다 중요합니다.

5.5, 객체가 자기 일을 알아서 하게 해봅시다

현재 `damage()`는 아직 조금 순진합니다.

```
fun damage(amount: Int) {  
    hp -= amount  
}
```

그래서 이런 것도 됩니다.

```
player.damage(-100)
```

피해를 -100만큼 입었더니 체력이 100 올라갔습니다.

공격자가 음수 데미지를 사용하는 순간 갑자기 성직자가 되어버렸네요.

의도한 기능이라면 괜찮겠지만 보통은 아닐 겁니다.

그러니 `Player`가 자기 규칙을 스스로 확인하게 해보죠.

```
fun damage(amount: Int) {  
    if (amount <= 0) {  
        return  
    }  
  
    hp -= amount  
}
```

이제 0 이하의 피해는 무시합니다.

체력이 음수가 되는 것까지 막고 싶다면 이렇게 할 수 있겠죠.

```
fun damage(amount: Int) {  
    if (amount <= 0) {  
        return  
    }  
  
    hp = maxOf(0, hp - amount)  
}
```

아무리 강한 공격을 받아도 HP는 최소 0에서 멈춥니다.

```
player.damage(999999999)
```

```
println(player.hp)
```

출력은,

0

이 되겠죠.

드디어 생과 사의 경계가 조금 명확해졌습니다.

철학과에 문의하지 않아도 되겠네요.

5.6, 왜 굳이 이렇게 귀찮게 하죠?

여기쯤 오면 이런 생각이 들 수도 있습니다.

그냥 외부 코드에서 알아서 정상적인 값만 넣으면 되는 거 아닌가요?

모든 개발자가 언제나 실수하지 않고, 모든 코드가 항상 의도대로 작성되며, 앞으로 프로젝트에 들어올 모든 사람이 여러분의 머릿속 설계를 완벽하게 이해한다면 그렇게 해도 됩니다.

참으로 훌륭한 계획입니다.

유니콘을 한 마리 잡아오시는 편이 더 쉬울지도 모르겠네요.

프로젝트는 커지고 사람도 늘어납니다.

그리고 무엇보다 가장 위험한 개발자가 한 명 있습니다.

3개월 뒤의 나입니다.

현재의 나는 분명히 기억하고 있습니다.

hp에는 이상한 값을 넣으면 안 돼.

그런데 3개월 뒤 새벽 2시 17분의 나는,

```
player.hp -= damage
```

같은 코드를 쓰면서 그 규칙을 기억하지 못할 가능성이 꽤 높습니다.

그러니 지켜야 할 규칙을 사람의 기억에만 맡기지 않고 코드 안에 넣는 겁니다.

```
fun damage(amount: Int) {  
    if (amount <= 0) return  
  
    hp = maxOf(0, hp - amount)  
}
```

이제 개발자가 규칙을 기억하고 있든 말든 같은 규칙이 항상 적용됩니다.

미래의 나를 믿지 마세요.

저도 저를 믿지 않습니다.

5.7, 객체가 자기 상태를 책임진다는 것

지금의 Player는 처음과 꽤 달라졌습니다.

예전에는 외부에서 이런 것이 가능했습니다.

```
player.hp = -999999999
```

이제는 불가능하죠.

```
class Player(  
    var name: String,  
    hp: Int,  
    var level: Int  
) {  
    var hp: Int = hp  
        private set  
  
    fun damage(amount: Int) {  
        if (amount <= 0) return  
  
        hp = maxOf(0, hp - amount)  
    }  
  
    fun heal(amount: Int) {  
        if (amount <= 0) return  
  
        hp += amount  
    }  
}
```

외부 코드는 더 이상 Player의 HP를 마음대로 고칠 수 없습니다.

대신 Player가 제공하는 행동을 통해 상태를 바꾸죠.

```
player.damage(10)  
player.heal(20)
```

그리고 그 과정에서 Player는 자기 규칙을 스스로 지킵니다.

hp는 0 미만이 되면 안 된다.

damage에는 양수만 들어와야 한다.

heal에도 양수만 들어와야 한다.

중요한 점은 이런 규칙이 프로젝트 곳곳에 흩어져 있지 않다는 것입니다.

Player의 상태를 관리하는 규칙이 Player 안에 모여 있죠.

지난 장에서는 데이터와 행동을 한곳에 묶었다면, 이번에는 그 데이터가 어떤 규칙으로 변경될 수 있는지까지 같은 곳에 묶고 있는 셈입니다.

점점 객체다운 객체가 되어가네요.

5.8, 이것을 캡슐화라고 부릅니다

이제 우리가 한 일에 이름을 붙여봅시다.

외부에서 상태를 직접 수정하지 못하게 막았습니다.

```
private set
```

상태를 바꾸는 방법은 객체가 제공하도록 만들었죠.

```
fun damage(amount: Int)
fun heal(amount: Int)
```

그리고 그 행동 내부에서 객체가 자기 상태에 대한 규칙을 지키게 했습니다.

이처럼 객체의 내부 상태나 구현을 외부에서 함부로 건드리지 못하게 하고, 필요한 기능을 정해진 통로를 통해 사용하게 만드는 것을 캡슐화(encapsulation)라고 부릅니다.

객체지향의 중요한 개념이라고 책마다 등장하는 그 캡슐화가 맞습니다.

이제야 이름을 알려드리네요.

우리는 정의를 먼저 외운 뒤 코드를 끼워 맞추는 것이 아니라, 코드를 직접 망쳐본 다음 왜 이런 개념이 필요해졌는지를 따라왔습니다.

개인적으로는 이쪽이 조금 덜 억울하다고 생각합니다.

5.9, 캡슐화는 **private** 도배 대회가 아닙니다

여기서 한 가지 주의할 점이 있습니다.

캡슐화를 배웠다고 모든 것 앞에 `private`을 붙이면 되는 것은 아닙니다.

```
class Player(
    private var name: String,
    private var hp: Int,
    private var level: Int
)
```

이렇게 전부 잠가버리면 아주 안전해 보입니다.

외부에서는 이름도 못 보고, 체력도 못 보고, 레벨도 못 봅니다.

외부에서 아무것도 못 하니 정말 안전하네요.

게임도 못 만듭니다.

캡슐화의 목적은 모든 정보를 무조건 숨기는 것이 아니라, 객체가 지켜야 하는 규칙과 내부 구현을 외부 코드가 함부로 깨뜨리지 못하게 만드는 것에 더 가깝습니다.

읽어도 되는 정보는 읽게 두고, 직접 변경하면 안 되는 것은 막고, 상태를 바꿔야 한다면 의미 있는 행동을 제공하면 되죠.

```
player.damage(10)
player.heal(20)
```

무조건 벽을 세우는 것이 아니라 필요한 출입구를 만드는 겁니다.
아무도 들어갈 수 없는 집은 보안성은 높을 수 있겠지만 생활하기
에는 조금 곤란하겠죠.

5.10, 잠시 정리해봅시다

처음에는 외부 코드가 객체의 상태를 마음대로 수정할 수 있었습니
다.

```
player.hp = -999999999
player.level = 999999999
```

그래서 직접 수정을 막았습니다.

```
var hp: Int = hp
    private set
```

대신 상태를 바꾸는 행동은 객체가 제공하게 했죠.

```
player.damage(10)
player.heal(20)
```

그리고 그 행동 안에서 객체가 지켜야 하는 규칙도 검사하게 만들
었습니다.

```
fun damage(amount: Int) {
    if (amount <= 0) return

    hp = maxOf(0, hp - amount)
}
```

결국 우리가 한 일은 이런 흐름입니다.

아무나 상태를 수정함

↓

문제가 생김

↓

직접 수정을 막음

↓

정해진 행동으로만 상태를 변경함

↓

객체가 자기 규칙을 스스로 지킴

이것이 캡슐화입니다.

객체에게 사생활을 주겠다고 시작했는데 생각보다 제법 중요한 원
칙이 하나 나왔네요.

그런데 이제 다른 문제가 보이기 시작합니다.

게임을 계속 만들다 보면 Player, Monster, NPC, Boss 같은 클래스가 잔뜩 생길 겁니다.

그리고 어느 날 코드를 열어보면 이런 모습을 발견할 수도 있겠죠.

```
class Player(  
    var name: String,  
    var hp: Int,  
    var x: Int,  
    var y: Int  
)
```

```
class Monster(  
    var name: String,  
    var hp: Int,  
    var x: Int,  
    var y: Int  
)
```

```
class NPC(  
    var name: String,  
    var hp: Int,  
    var x: Int,  
    var y: Int  
)
```

어라.

이거 어디서 많이 본 문제 아닌가요?

1장에서 변수들이 반복되던 문제가 이번에는 클래스 단위로 다시 나타났습니다.

함수가 반복될 때 함수를 만들어 재사용했듯이, 클래스도 어떤 방식으로 재사용할 수 있지 않을까요?

재사용의 재사용입니다.

슬슬 위험한 생각이 들기 시작하네요.

제 6 장, 함수가 탄생한 이유는 재사용성을 위해서였다. 그렇다면 클래스도 재사용할 수 있지 않을까? 재사용의 재사용. 그것이야말로 코딩의 극의!!!

지난 장 마지막에서 우리는 또 익숙한 광경을 발견했습니다.

```
class Player(  
    var name: String,  
    var hp: Int,  
    var x: Int,
```

```

        var y: Int
    )

class Monster(
    var name: String,
    var hp: Int,
    var x: Int,
    var y: Int
)

class NPC(
    var name: String,
    var hp: Int,
    var x: Int,
    var y: Int
)

```

이상 하죠.

분명 클래스라는 멋진 물건을 도입해서 반복을 줄였는데, 이제는 클래스 안에서 똑같은 코드가 반복되고 있습니다.

name, hp, x, y.

전부 똑같네요.

1장에서 변수들이 복제되기 시작했을 때와 별로 다르지 않습니다. 단지 문제의 크기가 변수에서 클래스로 승격됐을 뿐이죠.

버그도 승진을 합니다.

그렇다면 이런 생각을 해볼 수 있겠습니다.

함수는 같은 코드를 여러 번 쓰기 싫어서 만들었습니다.

클래스도 똑같은 이유로 재사용할 수 있지 않을까요?

재사용의 재사용입니다.

슬슬 위험한 냄새가 나네요.

6.1, 공통점을 위로 올려봅시다

Player, Monster, NPC를 다시 보죠.

세 클래스 모두 이름이 있고, 체력이 있고, 좌표를 가지고 있습니다.

그렇다면 이 공통된 부분에 이름을 하나 붙여봅시다.

Entity라고 하죠.

```

open class Entity(
    var name: String,
    var hp: Int,

```

```
    var x: Int,  
    var y: Int  
)
```

여기서 처음 보는 단어가 하나 있습니다.

open이네요.

코틀린의 클래스는 기본적으로 다른 클래스가 상속할 수 없도록 닫혀 있습니다.

그러니,

```
open class Entity
```

라고 써서,

이 클래스는 다른 클래스가 이어받아도 됩니다.

라고 허락해줘야 합니다.

왜 기본값이 닫혀 있는지는 나중에 다시 이야기하죠.

지금은 일단 문을 열어두겠습니다.

6.2, 조상님을 만들어봅시다

이제 Player가 Entity를 이어받게 해보죠.

```
class Player(  
    name: String,  
    hp: Int,  
    x: Int,  
    y: Int,  
    var level: Int  
) : Entity(  
    name,  
    hp,  
    x,  
    y  
)
```

조금 복잡해 보이지만 하는 일은 단순합니다.

Player가 자기에게만 필요한 level은 직접 가지고 있고,

name, hp, x, y는 Entity에게 맡긴 겁니다.

그러면 Player에서도 그대로 사용할 수 있습니다.

```
var player = Player(  
    name = "Yuhan",  
    hp = 100,  
    x = 10,
```

```

        y = 20,
        level = 1
    )

println(player.name)
println(player.hp)
println(player.x)
println(player.y)
println(player.level)

```

Player 안에 name을 직접 선언하지 않았는데도 사용할 수 있죠.

Entity가 가진 것을 Player가 물려받았기 때문입니다.

이것을 상속(**inheritance**)이라고 부릅니다.

드디어 족보가 생겼습니다.

6.3, 슬라임에게도 조상님이 생겼습니다

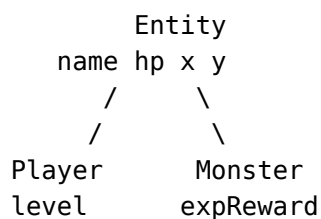
Monster도 같은 방식으로 만들 수 있겠죠.

```

class Monster(
    name: String,
    hp: Int,
    x: Int,
    y: Int,
    var expReward: Int
) : Entity(
    name,
    hp,
    x,
    y
)

```

이제 구조는 대략 이렇습니다.



Player도 Entity이고 Monster도 Entity입니다.

즉,

Player is an Entity.

Monster is an Entity.

라고 말할 수 있죠.

이 관계가 상당히 중요합니다.

상속은 단순히 코드 몇 줄 아끼는 문법이 아니라, 한 타입이 다른 타입의 한 종류라고 말하는 관계이기도 하거든요.

Player는 Entity의 한 종류입니다.

Monster도 Entity의 한 종류고요.

여기까지는 꽤 자연스럽습니다.

6.4, 행동도 물려받을 수 있겠죠?

데이터만 물려받을 이유는 없습니다.

모든 Entity가 피해를 받을 수 있다고 해보죠.

```
open class Entity(  
    var name: String,  
    hp: Int,  
    var x: Int,  
    var y: Int  
) {  
    var hp: Int = hp  
        private set  
  
    fun damage(amount: Int) {  
        if (amount <= 0) return  
  
        hp = maxOf(0, hp - amount)  
    }  
}
```

이제 Player에도 따로 damage()를 만들 필요가 없습니다.

```
var player = Player(  
    name = "Yuhan",  
    hp = 100,  
    x = 10,  
    y = 20,  
    level = 1  
)
```

```
player.damage(30)
```

```
println(player.hp)
```

출력은,

```
70
```

이 되겠죠.

Monster 역시 마찬가지입니다.

```
monster.damage(10)
```

이제 체력 처리 규칙도 Entity 한곳에만 있습니다.

아까 클래스마다 같은 코드를 복사하던 문제를 꽤 깔끔하게 해결했네요.

같은 코드를 세 번 썼다면 조상님을 만들어보십시오.

단, 정말 친척일 때만요.

이 마지막 문장이 중요합니다.

6.5, 아무거나 상속하면 안 됩니다

상속을 막 배운 사람에게는 세상이 갑자기 거대한 가게도로 보이기 시작합니다.

코드를 재사용할 수 있다니까 굉장히 편해 보이죠.

예를 들어 칼에도 이름이 있고 좌표가 있으니 이런 생각을 할 수도 있습니다.

```
class Sword(  
    name: String,  
    x: Int,  
    y: Int  
) : Entity(  
    name,  
    hp = 100,  
    x,  
    y  
)
```

컴파일은 가능하게 만들 수 있겠죠.

그런데 생각해봅시다.

칼은 정말 Entity인가요?

칼에게 HP가 있다는 게임이라면 그럴 수도 있습니다.

하지만 단순히 name, x, y 몇 줄을 재사용하고 싶다는 이유만으로 아무 관계나 상속으로 엮기 시작하면 이상한 족보가 만들어집니다.

```
Entity  
├─ Player  
├─ Monster  
├─ NPC  
├─ Sword  
├─ Door  
├─ Fireball  
├─ Inventory  
└─ DatabaseConnection
```

마지막쯤 가면 판타지 세계관이 아니라 개발자의 정신상태를 표현한 다이어그램에 가까워집니다.

상속을 사용할 때는 최소한,

A는 B의 한 종류인가?

라는 질문을 해보는 게 좋습니다.

Player는 Entity의 한 종류인가?

그렇듯하죠.

Monster는 Entity의 한 종류인가?

역시 그렇듯합니다.

DatabaseConnection은 Entity의 한 종류인가?

병원에 가보는 것이 좋겠습니다.

6.6, 그런데 플레이어만 다르게 행동하고 싶다면요?

모든 Entity가 같은 방식으로 피해를 받는다고 해봅시다.

그런데 Player는 방어력을 가지고 있어서 실제 피해량이 줄어들게 만들고 싶습니다.

그러면 부모에게서 받은 행동을 조금 바꿔야겠죠.

먼저 부모 쪽 함수를 open으로 만들어줍니다.

```
open class Entity(  
    var name: String,  
    hp: Int  
) {  
    var hp: Int = hp  
    protected set  
  
    open fun damage(amount: Int) {  
        if (amount <= 0) return  
  
        hp = maxOf(0, hp - amount)  
    }  
}
```

그리고 Player에서 다시 정의합니다.

```
class Player(  
    name: String,  
    hp: Int,  
    var defense: Int  
) : Entity(  
    name,
```

```

        hp
    ) {
        override fun damage(amount: Int) {
            val actualDamage = maxOf(0, amount - defense)

            hp = maxOf(0, hp - actualDamage)
        }
    }
}

```

override라는 단어가 보이네요.

부모가 가지고 있던 함수를 자식 쪽에서 다시 정의하는 것을 오버라이드(**override**)라고 합니다.

이제,

```

var player = Player(
    name = "Yuhan",
    hp = 100,
    defense = 3
)

```

```
player.damage(10)
```

을 실행하면 실제로는 7만큼의 피해를 받게 만들 수 있습니다.

상속받았지만 필요하다면 행동을 바꿀 수도 있는 거죠.

다만 코틀린은 이것도 아무 함수나 마음대로 바꾸게 하지 않습니다.

부모 쪽에서 open이라고 허락한 함수만 오버라이드할 수 있죠.

상속에도 출입통제가 있습니다.

지난 강의 영향을 아주 잘 받고 있네요.

6.7, 잠시 정리해봅시다

우리는 클래스 사이에서도 똑같은 코드가 반복되는 문제를 발견했습니다.

그래서 공통된 데이터와 행동을 Entity라는 클래스에 모았습니다.

```

Player  Monster  NPC
  \      |      /
   \     |     /
    \    |    /
     \   |   /
      \  |  /
       \ | /
        \|/
        Entity

```

그리고 각 클래스가 Entity를 상속하게 만들었죠.

이제 부모 클래스의 데이터와 행동을 자식 클래스가 물려받아 사용할 수 있습니다.

필요하다면 open된 행동을 override해서 자기 방식으로 바꿀 수도 있고요.

이것이 상속입니다.

코드를 재사용한다는 장점도 있지만, 그보다 더 중요한 것은 타입 사이에,

Player는 Entity의 한 종류다.

같은 관계를 만들 수 있다는 점입니다.

그런데 세상의 모든 관계가 “의 한 종류”인 것은 아니죠.

플레이어와 문은 친척이 아닙니다.

그런데 둘 다 공격받을 수는 있습니다.

그러면 어떻게 해야 할까요?

제 7 장, 형용사적 상속.

문이 하나 있습니다.

```
class Door(  
    var durability: Int  
)
```

문은 플레이어도 아니고 몬스터도 아니며, 우리가 만든 Entity라고 부르기도 조금 애매합니다.

그런데 게임 속에서는 문도 부술 수 있게 만들고 싶을 수 있죠.

```
door.damage(10)
```

플레이어도 피해를 받습니다.

```
player.damage(10)
```

벽도 부술 수 있는 게임이라면 벽도 마찬가지겠죠.

```
wall.damage(10)
```

셋은 전혀 다른 존재인데 한 가지 공통점이 있습니다.

피해를 받을 수 있습니다.

그렇다고 공통 조상을 하나 만들기는 조금 이상하죠.

```
    무언가  
  /   |   \  
Player Door Wall
```

이름부터 벌써 포기한 것 같습니다.

이럴 때 사용할 수 있는 다른 방법이 있습니다.

7.1, 명사가 아니라 형용사를 만들어봅시다

우리가 아까 만든 Entity는 대충 이런 질문에 답했습니다.

애는 무엇인가요?

플레이어는 Entity입니다.

몬스터도 Entity입니다.

어떤 대상의 정체를 표현하는 데 가까웠죠.

이번에는 질문을 조금 바꿔보겠습니다.

애는 무엇을 할 수 있나요?

플레이어는 피해를 받을 수 있습니다.

문도 피해를 받을 수 있고요.

벽도 피해를 받을 수 있습니다.

그러면 피해를 받을 수 있음이라는 능력 자체에 이름을 붙여보죠.

```
interface Damageable {  
    fun damage(amount: Int)  
}
```

interface.

새로운 단어가 나왔습니다.

지금은 일단 어떤 대상이 가져야 할 능력이나 약속을 표현하는 도구라고 생각해봅시다.

Damageable이라면,

너 Damageable이라고 했지?

그럼 damage()는 할 수 있어야 해.

라는 약속을 하는 셈이죠.

7.2, 플레이어는 Damageable합니다

Player가 이 인터페이스를 구현하게 만들어봅시다.

```
class Player(  
    var name: String,
```

```

        hp: Int
    ) : Damageable {

        var hp: Int = hp
        private set

        override fun damage(amount: Int) {
            if (amount <= 0) return

            hp = maxOf(0, hp - amount)
        }
    }
}

```

Damageable 인터페이스에서 damage()를 약속했기 때문에 Player는 그 함수를 실제로 구현해야 합니다.

그래서 override가 붙죠.

이제 Player는,

```

Player
├─ name
├─ hp
└─ Damageable
    └─ damage()

```

라고 생각할 수 있습니다.

Player는 플레이어라는 무엇이면서 동시에 피해를 받을 수 있는 성질도 가지고 있는 겁니다.

7.3, 문도 Damageable합니다

문에게도 같은 능력을 줘봅시다.

```

class Door(
    durability: Int
) : Damageable {

    var durability: Int = durability
    private set

    override fun damage(amount: Int) {
        if (amount <= 0) return

        durability = maxOf(0, durability - amount)
    }
}

```

이제 문도 피해를 받을 수 있습니다.

```
var door = Door(100)
```

```
door.damage(30)
```

```
println(door.durability)
```

출력은,

70

이 되겠죠.

중요한 것은 Door와 Player 사이에 상속 관계가 없다는 겁니다.

문은 플레이어가 아닙니다.

플레이어도 문이 아니고요.

둘의 공통 조상을 억지로 찾을 필요도 없습니다.

그저 둘 다 Damageable이라는 능력을 가지고 있을 뿐이죠.

오리도 날고 비행기도 납니다.

그렇다고 둘의 공통 조상을 찾기 시작하면 생물학과와 항공우주공학과가 동시에 화를 냅니다.

7.4, 그래서 왜 형용사적 상속인가요?

이 장 제목이 형용사적 상속이었죠.

물론 이것은 코틀린의 공식 용어가 아닙니다.

제가 또 지어냈습니다.

하지만 생각할 때 꽤 편한 비유입니다.

클래스 상속이,

Player는 Entity다.

처럼 명사와 명사를 연결하는 느낌이라면,

인터페이스는,

Player는 Damageable하다.

Door도 Damageable하다.

처럼 어떤 성질이나 능력을 붙이는 느낌이 강합니다.

그래서 입문 단계에서는 이렇게 생각해볼 수 있습니다.

class 상속

→ 애가 무엇인가?

interface 구현

→ 애가 무엇을 할 수 있는가?

다시 말하지만 엄밀한 언어학적 규칙은 아닙니다.

interface 이름이 반드시 형용사여야 하는 것도 아니고요.

그냥 머릿속에서 역할을 구분하기 위한 비유입니다.

책 한 권에 제가 지어낸 용어가 점점 많아지고 있네요.

나중에 용어사전이라도 만들어야 할지도 모르겠습니다.

귀찮으니 아마 안 만들 겁니다.

7.5, 능력은 여러 개여도 됩니다

코틀린 클래스는 직접적인 부모 클래스를 하나만 가질 수 있습니다.

가계도에서 친부모를 열일곱 명 둘 수는 없는 셈이죠.

그런데 인터페이스는 여러 개 구현할 수 있습니다.

```
interface Damageable {  
    fun damage(amount: Int)  
}
```

```
interface Movable {  
    fun move(dx: Int, dy: Int)  
}
```

```
interface Healable {  
    fun heal(amount: Int)  
}
```

플레이어에게 전부 붙여봅시다.

```
class Player(  
    var x: Int,  
    var y: Int,  
    hp: Int  
) : Damageable, Movable, Healable {  
  
    var hp: Int = hp  
        private set  
  
    override fun damage(amount: Int) {  
        if (amount <= 0) return  
  
        hp = maxOf(0, hp - amount)  
    }  
  
    override fun move(dx: Int, dy: Int) {
```

```

        x += dx
        y += dy
    }

    override fun heal(amount: Int) {
        if (amount <= 0) return

        hp += amount
    }
}

```

이제 Player는 이동할 수도 있고, 피해를 받을 수도 있고, 회복할 수도 있습니다.

문은 Damageable만 구현할 수도 있겠죠.

```

class Door(
    durability: Int
) : Damageable {
    // ...
}

```

서로 친척일 필요는 없습니다.

공통된 능력만 공유하면 되죠.

가족관계 대신 자격증을 발급한 셈입니다.

7.6, 인터페이스는 약속입니다

여기까지 오면 인터페이스를 이런 식으로 생각할 수 있습니다.

```

interface Damageable {
    fun damage(amount: Int)
}

```

이 코드는 구체적으로 HP를 어떻게 줄일지 정하지 않습니다.

그건 각 클래스가 알아서 합니다.

Player는 HP를 줄일 수도 있고,

```

override fun damage(amount: Int) {
    hp = maxOf(0, hp - amount)
}

```

Door는 내구도를 줄일 수도 있죠.

```

override fun damage(amount: Int) {
    durability = maxOf(0, durability - amount)
}

```

하지만 둘 다,

damage(amount)를 호출할 수 있다.

는 사실만큼은 보장됩니다.

이게 인터페이스가 주는 중요한 효과입니다.

내부가 어떻게 생겼는지는 달라도 밖에서 바라봤을 때 같은 약속을 지키게 만들 수 있죠.

이제 여기서 정말 재미있는 일이 하나 가능합니다.

플레이어인지 문인지 신경 쓰지 않고 그냥 Damageable한 무언가를 받을 수 있습니다.

그 이야기를 다음 장에서 해보죠.

제 8 장, 문맥이 중요한 이유.

현재 우리에게서 서로 전혀 다른 두 클래스가 있습니다.

```
class Player : Damageable {  
    override fun damage(amount: Int) {  
        // HP 감소  
    }  
}  
  
class Door : Damageable {  
    override fun damage(amount: Int) {  
        // 내구도 감소  
    }  
}
```

둘 다 Damageable입니다.

그렇다면 이런 함수도 만들 수 있겠죠.

```
fun hit(  
    target: Damageable,  
    amount: Int  
) {  
    target.damage(amount)  
}
```

이 함수는 Player를 받는다고 하지 않았습니까.

Door를 받는다고도 하지 않았고요.

그저,

Damageable한 거 하나 주세요.

라고 요구하고 있습니다.

그러면 정말 아무 Damageable이나 넘길 수 있습니다.

```
hit(player, 10)
hit(door, 10)
```

같은 함수입니다.

같은 damage() 호출이구요.

그런데 실제로 일어나는 일은 다릅니다.

슬슬 마법이 시작됩니다.

8.1, 같은 주문, 다른 마법

조금 더 분명하게 만들어봅시다.

```
interface Damageable {
    fun damage(amount: Int)
}
```

플레이어는 이렇게 구현합니다.

```
class Player(
    hp: Int
) : Damageable {

    var hp: Int = hp
    private set

    override fun damage(amount: Int) {
        hp = maxOf(0, hp - amount)

        println("플레이어가 피해를 입었습니다.")
    }
}
```

문은 이렇게 구현하고요.

```
class Door(
    durability: Int
) : Damageable {

    var durability: Int = durability
    private set

    override fun damage(amount: Int) {
        durability = maxOf(
            0,
            durability - amount
        )

        println("문이 부서지고 있습니다.")
    }
}
```

이제 실행해보죠.

```
hit(player, 10)
hit(door, 10)
```

첫 번째 호출에서는 `Player.damage()`가 실행됩니다.

두 번째에서는 `Door.damage()`가 실행되고요.

그런데 `hit()` 함수 안에는 이것밖에 없습니다.

```
target.damage(amount)
```

같은 코드인데 실제 `target`이 무엇이냐에 따라 실행되는 행동이 달라집니다.

같은 주문, 다른 마법입니다.

8.2, `target`은 `Damageable`인데 어떻게 알아요?

이 부분이 처음 보면 조금 신기합니다.

```
fun hit(
    target: Damageable,
    amount: Int
) {
    target.damage(amount)
}
```

`target`의 타입은 `Damageable`입니다.

그런데 실제로 이 변수는 `Player` 객체를 가리킬 수도 있고,

```
val target: Damageable = player
```

`Door` 객체를 가리킬 수도 있습니다.

```
val target: Damageable = door
```

코드를 작성하는 입장에서는 `target`이 `Damageable`이라는 사실만 알고 있습니다.

그러니 `Damageable`이 약속한 함수는 사용할 수 있죠.

```
target.damage(10)
```

그리고 실제 어느 `damage()`를 실행할지는 `target`이 실제로 가리키는 객체에 따라 결정됩니다.

`Player`를 가리키고 있으면 `Player.damage()`가,

`Door`를 가리키고 있으면 `Door.damage()`가 실행되죠.

문맥이 달라지면 같은 호출의 행동도 달라집니다.

이것이 이 장 제목이 문맥 이야기인 이유입니다.

8.3, 그래서 좋은 게 뭔데요?

별것 아닌 것처럼 느껴질 수도 있습니다.

그냥 Player와 Door를 따로 처리하면 되지 않나요?

물론 할 수 있습니다.

```
fun hitPlayer(  
    player: Player,  
    amount: Int  
) {  
    player.damage(amount)  
}
```

```
fun hitDoor(  
    door: Door,  
    amount: Int  
) {  
    door.damage(amount)  
}
```

벽이 추가되면요?

```
fun hitWall(...)
```

상자가 추가되면,

```
fun hitBox(...)
```

차가 추가되면,

```
fun hitCar(...)
```

축하합니다.

함수 이름 짓기 대회 시즌 2가 개최되었습니다.

우리는 이미 이런 일을 겪어봤죠.

그런데 Damageable을 사용하면 새로운 종류가 추가되어도 hit()은 고칠 필요가 없습니다.

```
fun hit(  
    target: Damageable,  
    amount: Int  
) {  
    target.damage(amount)  
}
```

새로운 클래스가 그저 Damageable을 구현하면 됩니다.

```
class Wall : Damageable {  
    override fun damage(amount: Int) {  
        // 벽의 피해 처리  
    }  
}
```

기존의 `hit()`은 그대로 쓸 수 있죠.

```
hit(wall, 10)
```

기존 코드를 뜯어고치지 않고 새로운 대상을 끼워넣을 수 있게 됐습니다.

제법 강력합니다.

8.4, 이것이 다형성입니다

이제 또 이름을 붙일 시간입니다.

우리는 하나의 `Damageable`이라는 타입을 통해 여러 종류의 객체를 다뤘습니다.

```
val a: Damageable = player
val b: Damageable = door
val c: Damageable = wall
```

그리고 똑같이,

```
damageable.damage(10)
```

이라고 호출해도 실제 객체에 따라 서로 다른 행동이 실행됐죠.

이처럼 하나의 공통된 타입이나 인터페이스를 통해 여러 구체적인 형태를 다룰 수 있는 성질을 다형성(**polymorphism**)이라고 부릅니다.

말 그대로 여러 형태라는 뜻입니다.

프로그래밍 용어치고 이름이 꽤 솔직하네요.

하나의 `Damageable`이라는 얼굴 뒤에,

```
Player
Door
Wall
Monster
Vehicle
```

같은 여러 형태가 들어갈 수 있습니다.

그리고 호출하는 쪽은 구체적으로 무엇이 들어왔는지 몰라도 됩니다.

그저,

`Damageable`이면 `damage()`를 할 수 있겠지.

라고 믿고 호출하면 됩니다.

지난 장의 인터페이스가 약속이었다면, 다형성은 그 약속을 믿고 코드를 작성할 수 있게 해주는 힘에 가깝습니다.

8.5, 구체적인 것보다 약속을 바라봅시다

아까 만든 `hit()`을 다시 보죠.

```
fun hit(  
    target: Damageable,  
    amount: Int  
) {  
    target.damage(amount)  
}
```

여기에는 `Player`라는 단어도 없고 `Door`라는 단어도 없습니다.

구체적인 구현을 직접 바라보지 않고 `Damageable`이라는 약속만 바라보고 있죠.

덕분에 `hit()`은 앞으로 추가될 클래스도 사용할 수 있습니다.

아직 존재하지 않는 클래스까지요.

미래의 코드와도 호환되는 셈입니다.

물론 미래 예지 능력이 생긴 것은 아닙니다.

그냥 의존하는 정보의 양을 줄인 겁니다.

`hit()`이 알고 있는 것은,

 애는 `damage()`를 할 수 있다.

하나뿐입니다.

알아야 할 것이 적어지면 서로 얽히는 것도 줄어듭니다.

그리고 앞으로 객체지향 설계를 계속 공부하다 보면 이 서로 얼마나 알고 있는가라는 문제가 상당히 자주 등장할 겁니다.

그건 다음 권의 저자에게 떠넘기죠.

지금의 저는 이미 충분히 일했습니다.

8.6, 잠시 정리해봅시다

우리는 `Player`, `Door`, `Wall`처럼 서로 전혀 다른 객체들에게 `Damageable`이라는 공통된 약속을 붙였습니다.

그리고 함수는 구체적인 클래스 대신 그 약속을 받게 만들었죠.

```
fun hit(  
    target: Damageable,
```

```
        amount: Int
    )
```

덕분에 호출하는 코드는 대상이 정확히 무엇인지 몰라도 됩니다.
같은 호출을 해도 실제 객체에 맞는 구현이 실행되고요.

```
target.damage(10)
```

```
Player → HP 감소
Door   → 내구도 감소
Wall   → 벽 체력 감소
```

이것이 다형성입니다.

하나의 리모컨으로 여러 기계를 조종하는 흑마술이라고 생각해도 됩니다.

물론 리모컨 버튼 규격, 그러니까 인터페이스는 맞아야겠죠.

그런데 이제 또 다른 문제가 하나 있습니다.

상속도 배웠고 인터페이스도 배웠으니 신나서 모든 것을 상속으로 연결하기 시작하면 어떻게 될까요?

음.

죽보가 아키텍처가 됩니다.

제 9 장, 코딩은 레고임.

상속은 꽤 강력합니다.

공통된 상태와 행동을 부모 클래스에 올려놓고 여러 자식이 물려받을 수 있죠.

그런데 강력한 도구를 배우면 인간은 보통 그것을 모든 곳에 쓰고 싶어집니다.

망치를 들면 세상이 못으로 보이고,

상속을 배우면 세상이 가족관계로 보입니다.

예를 들어 플레이어에게 여러 기능이 있다고 해봅시다.

플레이어는 움직일 수 있고, 체력도 있고, 인벤토리도 있고, 마나도 있고, 상태이상도 받으며, 저장도 되어야 합니다.

전부 상속으로 표현하려 들면 슬슬 이런 생각을 하게 됩니다.

```
Entity
  ↓
LivingEntity
  ↓
MovableLivingEntity
```

```
↓
DamageableMovableLivingEntity
↓
InventoryDamageableMovableLivingEntity
↓
Player
```

클래스 이름만 읽다가 수명이 줄어들 것 같습니다.
다른 방법을 찾아보죠.

9.1, 플레이어가 체력을 상속받아야만 할까요? 체력이라는 기능을 하나의 독립된 부품으로 만들어봅시다.

```
class Health(
    current: Int,
    val max: Int
) {
    var current: Int = current
    private set

    fun damage(amount: Int) {
        if (amount <= 0) return

        current = maxOf(
            0,
            current - amount
        )
    }

    fun heal(amount: Int) {
        if (amount <= 0) return

        current = minOf(
            max,
            current + amount
        )
    }
}
```

이제 Player가 Health를 상속하지 않고 그냥 가지고 있게 해보죠.

```
class Player(
    val name: String,
    val health: Health
)
```

사용할 때는,

```
var player = Player(
    name = "Yuhan",
    health = Health(
        current = 100,
```

```
        max = 100
    )
)
```

```
player.health.damage(10)
```

이라고 할 수 있습니다.

Player는 Health가 아닙니다.

그 대신 Health를 가지고 있죠.

Player is a Health.

는 이상하지만,

Player has a Health.

는 꽤 자연스럽습니다.

이런 식으로 객체 안에 다른 객체를 부품처럼 포함해서 기능을 구성하는 것을 합성(**composition**)이라고 부릅니다.

9.2, 위치도 부품으로 만들어봅시다

좌표도 비슷합니다.

```
class Position(
    var x: Int,
    var y: Int
) {
    fun move(
        dx: Int,
        dy: Int
    ) {
        x += dx
        y += dy
    }
}
```

그러면 플레이어는 여러 부품을 조립해서 만들 수 있죠.

```
class Player(
    val name: String,
    val health: Health,
    val position: Position
)
```

생성할 때는,

```
var player = Player(
    name = "Yuhan",
```

```

    health = Health(
        current = 100,
        max = 100
    ),
    position = Position(
        x = 10,
        y = 20
    )
)

```

이제 플레이어는 체력 부품과 위치 부품을 가지고 있습니다.

```

player.health.damage(10)
player.position.move(3, -1)

```

그리고 몬스터도 같은 부품을 쓸 수 있겠죠.

```

class Monster(
    val health: Health,
    val position: Position
)

```

문은 Health 비슷한 부품만 가지고 위치는 고정할 수도 있고요.

필요한 부품만 꽂으면 됩니다.

가족관계를 만들 필요가 없죠.

9.3, 상속은 유전자고 합성은 레고입니다

상속을 하면 부모와 자식의 관계가 만들어집니다.

한번 정해지면 꽤 강하게 묶이죠.

```

Entity
  ↓
Player

```

반면 합성은 필요한 객체를 부품으로 가지고 있는 형태입니다.

```

Player
├─ Health
├─ Position
└─ Inventory

```

입문 단계의 비유로 생각하면,

상속
→ 유전자

합성
→ 레고 블록

정도라고 볼 수 있습니다.

물론 실제 프로그래밍 언어의 의미가 정말 유전자와 레고라는 뜻은 아닙니다.

제가 또 비유를 만들었습니다.

하지만 차이는 꽤 잘 드러납니다.

상속은,

나는 이것의 한 종류다.

라는 관계에 가깝고,

합성은,

나는 이것을 가지고 있다.

라는 관계에 가깝습니다.

둘 중 어느 것이 무조건 더 좋은 것은 아닙니다.

관계가 다릅니다.

Player가 Entity의 한 종류라는 표현이 자연스럽다면 상속을 생각할 수 있고,

Player가 Inventory를 가지고 있다는 표현이 자연스럽다면 합성이 더 맞겠죠.

인벤토리를 상속받은 인간을 만들기 시작하면 설정이 조금 무서워집니다.

9.4, 그런데 `player.health.damage()`가 길어요

합성에도 불편한 점은 있습니다.

이런 코드가 계속 생길 수 있죠.

```
player.health.damage(10)
player.health.heal(20)
```

우리는 그냥,

```
player.damage(10)
```

이라고 쓰고 싶을 수도 있습니다.

그러면 Player가 요청을 받아서 내부의 Health에게 넘겨주면 됩니다.

```
class Player(
    val name: String,
    private val health: Health
) {
```

```

    fun damage(amount: Int) {
        health.damage(amount)
    }

    fun heal(amount: Int) {
        health.heal(amount)
    }
}

```

이제 밖에서는,

```
player.damage(10)
```

이라고 부르면 됩니다.

실제 처리는 Player가 직접 하지 않고 Health에게 맡기고 있죠.

이렇게 어떤 일을 다른 객체에게 넘겨서 처리하게 하는 것을 위임(**delegation**)이라고 합니다.

말 그대로 일을 위임한 겁니다.

회사에서 많이 보던 개념이네요.

다행히 여기서는 책임 떠넘기기가 설게 기법입니다.

9.5, 코틀린은 위임 문법까지 있습니다

코틀린은 위임을 자주 사용하는 언어라 아예 문법도 제공합니다.

아까의 Damageable을 다시 가져와보죠.

```

interface Damageable {
    fun damage(amount: Int)
}

```

Health가 이를 구현하게 만들 수 있습니다.

```

class Health(
    hp: Int
) : Damageable {

    var hp: Int = hp
    private set

    override fun damage(amount: Int) {
        if (amount <= 0) return

        hp = maxOf(0, hp - amount)
    }
}

```

그리고 Player는 이렇게 쓸 수 있습니다.

```
class Player(  
    private val health: Health  
) : Damageable by health
```

처음 보면 조금 수상합니다.

Damageable by health

하지만 뜻은 상당히 단순합니다.

Player도 Damageable이라고 할 건데,

Damageable의 실제 구현은 health에게 맡길게요.

정도입니다.

그러면 직접 damage()를 다시 작성하지 않아도,

```
player.damage(10)
```

이 가능합니다.

코틀린이 중간 전달 코드를 대신 만들어주는 셈이죠.

개발자가 귀찮아할 미래를 예측한 언어입니다.

훌륭합니다.

9.6, 상속으로 모든 걸 해결하려는 자의 최후

상속이 나쁘다는 이야기는 아닙니다.

상속은 아주 유용합니다.

다만 관계가 맞지 않는데 단순히 코드 재사용만을 위해 상속을 사용하기 시작하면 문제가 생기기 쉽습니다.

예를 들어 게임 캐릭터를 만들면서,

```
Entity  
↓  
LivingEntity  
↓  
MovingEntity  
↓  
InventoryEntity  
↓  
MagicEntity  
↓  
Player
```

를 만들었다고 해봅시다.

그런데 움직이지 않는 상인이 인벤토리는 가져야 한다면요?

날아다니지만 생명체는 아닌 드론이 생기면요?
체력은 있는데 위치가 없는 글로벌 보스가 생기면요?
처음에는 완벽해 보였던 가계도에 예외가 하나씩 끼어듭니다.
그리고 개발자는 새벽 세 시에 클래스 다이어그램을 바라보며,

내가 어디서부터 잘못했지?

라고 생각하게 됩니다.

대체로 꽤 오래전입니다.

필요한 기능을 독립된 부품으로 만들고 조립할 수 있다면 이런 상황을 피하기 쉬워집니다.

그래서 흔히,

상속보다 합성을 선호하라.

라는 조언을 볼 수 있습니다.

이 말 역시 “상속을 절대 쓰지 마라”라는 뜻은 아닙니다.

정확히는 단순한 코드 재사용만이 목적이라면 합성을 먼저 생각해 보라에 가깝습니다.

전기톱도 강력한 도구입니다.

그렇다고 샌드위치를 전기톱으로 자르지는 않죠.

9.7, 잠시 정리해봅시다

우리는 지금까지 객체들을 서로 연결하는 방법을 두 가지 봤습니다.

상속

Player

↓

Entity

Player is an Entity.

그리고,

합성

Player

├ Health

├ Position

└ Inventory

Player has a Health.

Player has a Position.

Player has an Inventory.

상속은 타입 사이의 관계를 표현하고,

합성은 객체를 여러 부품으로 조립하게 해줍니다.

그리고 어떤 부품에게 실제 일을 맡기는 위임도 살펴봤죠.

이제 객체를 만들고, 감추고, 물려받고, 약속을 만들고, 여러 구현을 같은 타입으로 다루고, 부품으로 조립하는 데까지 왔습니다.

여기까지 왔는데 한 가지 문제가 있습니다.

우리는 아직 객체지향 프로그래밍이 무엇인지 제대로 정의하지 않았습니 다.

9장이나 왔는데요.

이제는 도망갈 수 없습니다.

마지막 장입니다.

제 10 장, 객체지향에 대하여.

자, 드디어 여기까지 왔습니다.

객체지향 프로그래밍을 설명하는 책에서 10장에 와서야 객체지향 프로그래밍이 무엇인지 설명하겠습니다.

교재 구성에 문제가 있어 보인다고요?

반대로 생각해 주세요.

지금까지 객체지향을 설명하지 않은 것이 아닙니다.

직접 만들게 했을 뿐입니다.

1장에서 우리는 변수들이 흩어져 있는 불편함을 겪었습니다.

2장에서는 관련된 데이터를 하나로 묶었죠.

3장에서는 데이터와 행동을 함께 두었습니다.

4장에서는 클래스와 객체가 무엇인지 살펴봤고,

5장에서는 객체가 자기 상태를 지키게 만들었습니다.

6장에서는 타입 사이의 공통점을 상속으로 표현했고,

7장에서는 인터페이스로 능력과 약속을 표현했습니다.

8장에서는 같은 약속을 지키는 여러 객체를 하나의 타입으로 다뤘고,

9장에서는 객체들을 부품처럼 조립하고 일을 위임했습니다.

이제 이름만 붙이면 됩니다.

10.1, 그래서 객체지향이 뭔데요?

아주 거칠게 말하면 객체지향 프로그래밍은,

프로그램을 서로 상태와 행동을 가진 객체들의 상호작용으로 구성하는 방식

이라고 생각할 수 있습니다.

게임을 예로 들면,

Player
Monster
Door
Inventory
Weapon
Health
Position

같은 객체들이 있고,

서로 메시지를 주고받듯 행동을 호출합니다.

```
player.attack(monster)  
monster.damage(10)
```

```
player.open(door)
```

```
inventory.add(item)
```

각 객체는 자기가 맡은 상태와 행동을 가지고 있고, 다른 객체는 필요한 기능을 요청합니다.

모든 데이터를 한곳에 쌓아놓고 거대한 함수 하나가 전부 관리하는 대신, 프로그램의 책임을 여러 객체에 나누는 셈이죠.

사람이 일을 나누면 조직이 되고,

코드가 일을 나누면 아키텍처가 됩니다.

잘 나누면요.

잘못 나누면 회의가 늘어납니다.

코드도 비슷합니다.

10.2, 객체 지향은 **class** 많이 쓰기 대회가 아닙니다

객체 지향을 막 배우면 이런 생각을 하기 쉽습니다.

객체 지향적으로 만들려면 모든 걸 클래스로 만들어야 하나요?

아닙니다.

예를 들어 두 수를 더하는 함수가 있다고 해보죠.

```
fun add(  
    a: Int,  
    b: Int  
): Int {  
    return a + b  
}
```

굳이 이런 식으로 만들 필요는 없습니다.

```
class AdditionManagerFactoryService {  
    fun createAdditionOperation(  
        a: Int,  
        b: Int  
    ): AdditionOperation {  
        // ...  
    }  
}
```

이쯤 되면 숫자 두 개 더하려다가 기업용 솔루션이 탄생합니다.

객체 지향의 목적은 클래스를 많이 만드는 것이 아닙니다.

프로그램에서 함께 움직이는 상태와 행동, 그리고 책임을 적절한 객체에 배치하는 것이죠.

함수가 적합하면 함수를 쓰면 됩니다.

값 하나면 충분하면 값 하나를 쓰고요.

Kotlin에서는 객체 지향 코드와 함수형 스타일, 절차적인 코드를 얼마든지 섞어서 사용할 수 있습니다.

도구는 목적이 아닙니다.

망치를 샀다고 집 안의 모든 물건을 두드릴 필요는 없죠.

10.3, 그 유명한 캡슐화, 상속, 다형성

객체 지향을 검색하다 보면 흔히 몇 가지 단어가 등장합니다.

캡슐화

상속

다형성

추상화

가끔 이것들을 객체지향의 네 가지 특징 또는 네 가지 기둥이라고 부르기도 하죠.

우리는 앞의 세 개를 이미 직접 만났습니다.

캡슐화는 객체가 자기 상태와 규칙을 지키게 하면서 등장했습니다.

```
var hp: Int = hp
    private set
```

상속은 여러 클래스의 공통된 관계를 표현하다가 등장했구요.

```
class Player(...) : Entity(...)
```

다형성은 여러 객체를 같은 약속으로 다루면서 등장했습니다.

```
fun hit(target: Damageable) {
    target.damage(10)
}
```

그런데 네 번째인 추상화는 뭘까요?

사실 이것도 계속 사용하고 있었습시다.

이름만 안 붙였을 뿐이죠.

10.4, 추상화는 지금 몰라도 되는 것을 치우는 일입니다
우리가 이런 코드를 사용한다고 해보죠.

```
player.damage(10)
```

이 코드를 호출하는 쪽은 Player 내부에서 HP가 어떻게 저장되는지 몰라도 됩니다.

```
var hp: Int
```

인지,

```
private val health: Health
```

인지도 꼭 알 필요가 없고요.

피해를 계산할 때 방어력 공식을 어떻게 적용하는지도 호출하는 쪽에서는 신경 쓰지 않을 수 있습니다.

그저,

damage(10)을 호출하면 이 객체가 10만큼의 피해를 적절히 처리한다.

는 사실만 알면 되죠.

복잡한 세부사항 중 지금 필요한 부분만 드러내고 나머지는 뒤로 치우는 것입니다.

이런 과정을 추상화(**abstraction**)라고 부릅니다.

추상화는 어려운 것을 멋진 말로 바꾸는 기술이 아닙니다.

지금 알 필요 없는 것을 치우는 기술에 가깝죠.

자동차를 운전하기 위해 엔진의 점화 시기와 연료 분사량을 매 순간 직접 계산할 필요가 없는 것과 비슷합니다.

핸들과 페달이라는 인터페이스만 알면 됩니다.

엔진룸을 계속 열고 운전하면 많이 불편하겠죠.

10.5, 우리는 객체에게 무엇을 말했나요?

처음의 코드를 다시 생각해봅시다.

```
var playerHp = 100
var playerLevel = 1
var playerGold = 500
```

상태는 전부 밖에 있었습니다.

상태를 바꾸는 코드도 밖에 있었어요.

```
playerHp -= 10
playerLevel += 1
playerGold += 100
```

프로그램이 커질수록 모든 코드가 모든 데이터를 알고 직접 건드리게 됩니다.

우리는 조금씩 책임을 옮겼습니다.

```
player.damage(10)
player.heal(20)
player.gainExp(100)
```

이제 외부 코드는,

HP를 정확히 몇으로 바꿔야 하지?

를 고민하지 않습니다.

그저,

플레이어가 10의 피해를 받았다.

라는 사건만 전달합니다.

실제 상태를 어떻게 바꿀지는 Player가 책임집니다.

이게 꽤 중요한 변화입니다.

객체 지향을 단순히 “데이터와 함수를 클래스 안에 넣는 방법”이라고만 생각하면 이 부분을 놓치기 쉽습니다.

더 중요한 것은,

누가 무엇을 알고, 누가 무엇을 책임질 것인가.

에 가깝습니다.

객체는 단순한 데이터 상자가 아니라 자기 역할을 맡은 작은 단위가 되는 거죠.

10.6, 객체 지향은 종교가 아닙니다

여기까지 읽고 나면 객체 지향이 모든 문제를 해결하는 엄청난 기술처럼 느껴질 수도 있습니다.

그렇지는 않습니다.

객체 지향으로 짜면 오히려 더 복잡해지는 문제도 있고, 순수 함수 몇 개로 끝내는 편이 훨씬 좋은 문제도 있습니다.

데이터 변환 작업이라면 함수형 스타일이 더 자연스러울 수도 있고,

아주 단순한 프로그램이라면 절차적으로 작성하는 편이 읽기 쉬울 수도 있죠.

객체 지향은 문제를 바라보는 여러 방법 중 하나입니다.

꽤 강력하고, 꽤 널리 사용되고, 상태와 책임이 복잡하게 얽힌 프로그램에서 특히 유용한 방법이지만 절대적인 정답은 아닙니다.

OOP는 종교가 아닙니다.

상속도 교리가 아니고,

SOLID도 십계명이 아닙니다.

망치를 들었다고 세상이 못으로 보이면 큰일 납니다.

어떤 방식이 지금 문제를 가장 단순하고 명확하게 표현하는지를 먼저 생각하는 편이 좋습니다.

객체 지향을 제대로 이해한다는 것은 모든 곳에 객체 지향을 사용하는 것이 아니라, 언제 객체가 도움이 되는지를 판단할 수 있게 되는 것에 더 가깝습니다.

10.7, 그래서 우리가 지금까지 한 일은 무엇이이었을까요?

처음에는 변수들이 있었었습니다.

```
name  
hp
```

```
level
gold
x
y
```

흩어져 있으니 묶었습니다.

```
Player
├─ name
├─ hp
├─ level
├─ gold
├─ x
└─ y
```

행동도 같이 다니니 그것도 넣었습니다.

```
Player
├─ 상태
│   ├── name
│   ├── hp
│   └─ level
└─ 행동
    ├── damage()
    ├── heal()
    └─ levelUp()
```

아무나 상태를 바꾸니 문을 잠갔습니다.

```
외부 코드
├─ X → hp 직접 수정
└─ 0 → damage()
      heal()
```

비슷한 객체가 생기니 공통점을 묶었고,

```
      Entity
     /    \
  Player  Monster
```

친척은 아니지만 같은 능력을 가진 객체가 생기니 인터페이스를 만들었습니다.

```
Player ┌
Door   ├── Damageable
Wall   └─
```

같은 인터페이스로 서로 다른 객체를 다루면서 다형성이 생겼고, 상속으로 표현하기 어색한 기능은 객체를 부품처럼 조립했습니다.

```
Player
├─ Health
```

└ Position
└ Inventory

처음부터,

자, 객체지향의 네 가지 특징을 외우세요.

라고 시작하지 않았습니다.

불편한 코드를 하나씩 고치다 보니 여기까지 왔죠.

우리가 객체지향을 배웠다가보다,

필요에 따라 객체지향을 다시 발명한 셈입니다.

10.8, 이제야 정의를 드리겠습니다

그러면 이 책에서 사용할 정의를 마지막으로 하나 남겨봅시다.

객체지향 프로그래밍은 상태와 행동, 그리고 그에 대한 책임을 객체라는 단위로 묶고, 객체들이 서로 협력하도록 프로그램을 구성하는 방식입니다.

완벽하게 모든 객체지향 언어와 모든 학자의 관점을 포괄하는 우주의 절대적 정의는 아닙니다.

그런 정의를 제가 여기서 발명했다면 이 책을 무료 PDF로 뿌리고 있을 상황이 아닐 겁니다.

하지만 지금까지 우리가 만든 구조를 설명하기에는 충분합니다.

중요한 것은 문장을 외우는 것이 아닙니다.

왜 데이터와 행동을 묶었는지,

왜 외부에서 상태를 함부로 만지지 못하게 했는지,

왜 공통된 약속을 만들었는지,

왜 상속과 합성을 구분해야 하는지,

그리고 왜 객체에게 자기 일을 맡겼는지를 이해하는 것이죠.

그걸 이해했다면 객체지향이라는 단어의 정의를 조금 다르게 외워도 크게 문제되지 않습니다.

개념을 알고 있으니깐요.

10.9, 끝났나요?

네.

적어도 이 책은요.

잘 따라오셨습니다.

우리는 변수 몇 개에서 출발해서 클래스, 객체, 캡슐화, 상속, 인터페이스, 다형성, 합성, 위임까지 왔습니다.

처음 코드는 이랬습니다.

```
var playerHp = 100
```

지금은 이 정도까지 생각할 수 있게 됐죠.

```
player.damage(10)
```

겉보기에는 오히려 짧아졌습니다.

그 짧은 코드 뒤에 누가 상태를 관리하고, 어떤 규칙을 지키고, 어떤 약속을 따르며, 무엇을 다른 객체에게 맡기는지가 들어가 있을 뿐입니다.

좋은 추상화는 사용하는 쪽을 심심하게 만듭니다.

대충 호출했는데 알아서 잘 돌아가죠.

그리고 어떻게 돌아가는지 모르겠다고요?

제가 그럴 때마다 쓰는 커밋 메시지가 하나 있습니다.

```
I DONT KNOW HOW IT WORKS.
```

받아적으세요.

물론 협업 프로젝트에서는 그러지 마세요.

미래의 동료가 여러분을 찾으러 올 수도 있습니다.

10.10, 그런데 코틀린은 아직 많이 남았습니다

여기까지 배웠다고 코틀린을 전부 배운 것은 아닙니다.

제네릭도 남았고,

sealed class도 남았고,

고차 함수와 람다도 있고,

코루틴도 있으며,

컬렉션 API도 있고,

리플렉션도 있고,

DSL도 있고,

타입 시스템 이야기도 한참 남아 있습니다.

객체 설계만 보더라도 의존성, 결합도, 응집도, SOLID, 불변성, 테스트 가능성 같은 이야기가 줄을 서서 기다리고 있죠.

하지만 그건 다음 이야기입니다.

여기서는 객체가 왜 생겨났고, 왜 자기 상태를 지키며, 왜 서로 다른 객체를 같은 약속으로 다루고, 왜 때로는 상속 대신 조립해야 하는지까지 이해했으면 충분합니다.

나머지가 궁금하시다고요?

저술하기 귀찮음으로 Kotlin in Action 같은 좋은 책으로 가세요.

왜요.

뭐.

저도 쉬어야 할 거 아닙니까.